

FinalProjectSTA4724

December 6, 2025

1 STA 4742 Final Project

Group 5 Members: David Alvarado, Joaquin Hidalgo, Guadalupe Molinari, Matheus Santos, Natiana Urdaneta

1.0.1 Import libraries

```
[686]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, label_binarize
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc,
    roc_auc_score,
    precision_recall_curve,
    average_precision_score
)

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neighbors import NearestNeighbors
from sklearn import neighbors
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

```
[687]: RANDOM_STATE = 42
```

1.0.2 Load Dataset

```
[689]: df = pd.read_csv("Sleep_health_and_lifestyle_dataset.csv")
```

```
[690]: df_clust = df.copy() #we will use it for k-means
```

```
[691]: print("Shape:", df.shape)
print("Columns:", list(df.columns))
print("\nInfo:")
print(df.info())
print("\nMissing values:\n", df.isnull().sum())
```

Shape: (374, 13)

Columns: ['Person ID', 'Gender', 'Age', 'Occupation', 'Sleep Duration', 'Quality of Sleep', 'Physical Activity Level', 'Stress Level', 'BMI Category', 'Blood Pressure', 'Heart Rate', 'Daily Steps', 'Sleep Disorder']

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 374 entries, 0 to 373

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	Person ID	374 non-null	int64
1	Gender	374 non-null	object
2	Age	374 non-null	int64
3	Occupation	374 non-null	object
4	Sleep Duration	374 non-null	float64
5	Quality of Sleep	374 non-null	int64
6	Physical Activity Level	374 non-null	int64
7	Stress Level	374 non-null	int64
8	BMI Category	374 non-null	object
9	Blood Pressure	374 non-null	object
10	Heart Rate	374 non-null	int64
11	Daily Steps	374 non-null	int64
12	Sleep Disorder	155 non-null	object

dtypes: float64(1), int64(7), object(5)

memory usage: 38.1+ KB

None

Missing values:

Person ID	0
Gender	0
Age	0
Occupation	0
Sleep Duration	0
Quality of Sleep	0
Physical Activity Level	0

```

Stress Level          0
BMI Category          0
Blood Pressure        0
Heart Rate            0
Daily Steps           0
Sleep Disorder        219
dtype: int64

```

1.0.3 Preliminary Data Cleaning

```

[693]: # Fill sleep disorder NA values
df["Sleep Disorder"] = df["Sleep Disorder"].fillna("No Disorder")

df = df.drop(columns=["Person ID"])

# Normalize BMI categories, unify 'Normal' vs 'Normal Weight'
df["BMI Category"] = df["BMI Category"].replace({"Normal Weight": "Normal"})

[694]: print("\nAfter cleaning:")
print(df.info())

```

After cleaning:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 374 entries, 0 to 373
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Gender                                374 non-null    object
1   Age                                   374 non-null    int64
2   Occupation                            374 non-null    object
3   Sleep Duration                        374 non-null    float64
4   Quality of Sleep                      374 non-null    int64
5   Physical Activity Level               374 non-null    int64
6   Stress Level                          374 non-null    int64
7   BMI Category                          374 non-null    object
8   Blood Pressure                        374 non-null    object
9   Heart Rate                           374 non-null    int64
10  Daily Steps                           374 non-null    int64
11  Sleep Disorder                        374 non-null    object
dtypes: float64(1), int64(6), object(5)
memory usage: 35.2+ KB
None

```

1.0.4 Exploratory Data Analysis

```
[696]: cat_cols = df.select_dtypes(include="object").columns.tolist()
num_cols = df.select_dtypes(include=np.number).columns.tolist()
```

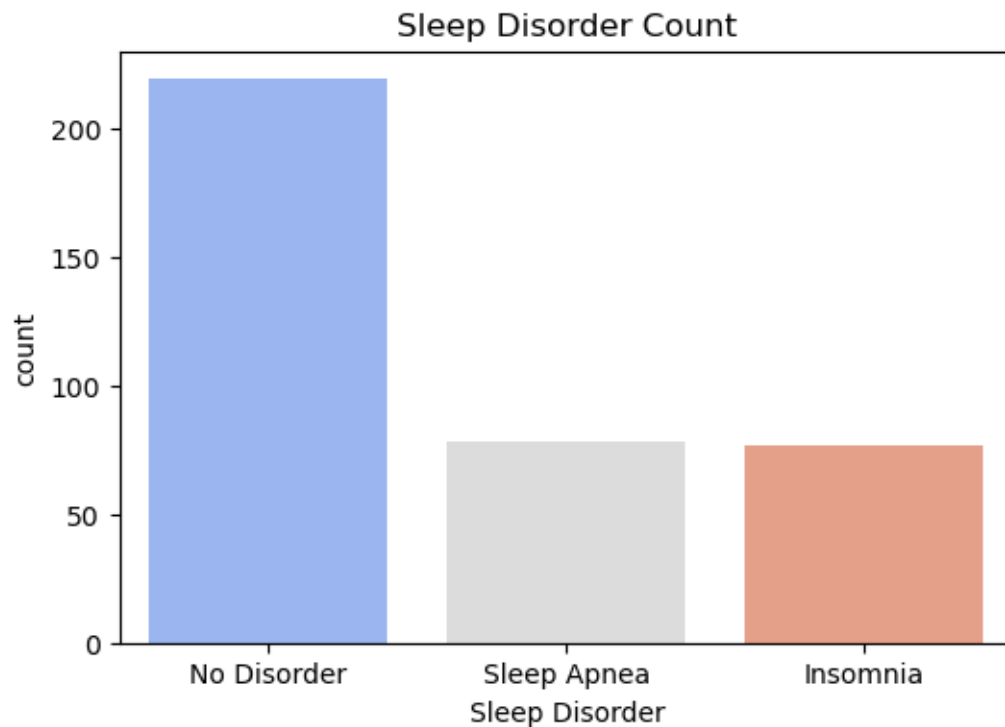
```
[697]: plt.figure(figsize=(6,4))
sns.countplot(x=df["Sleep Disorder"], palette="coolwarm")
plt.title("Sleep Disorder Count")
plt.show()
```

C:\Users\molin\AppData\Local\Temp\ipykernel_39696\3354542254.py:2:

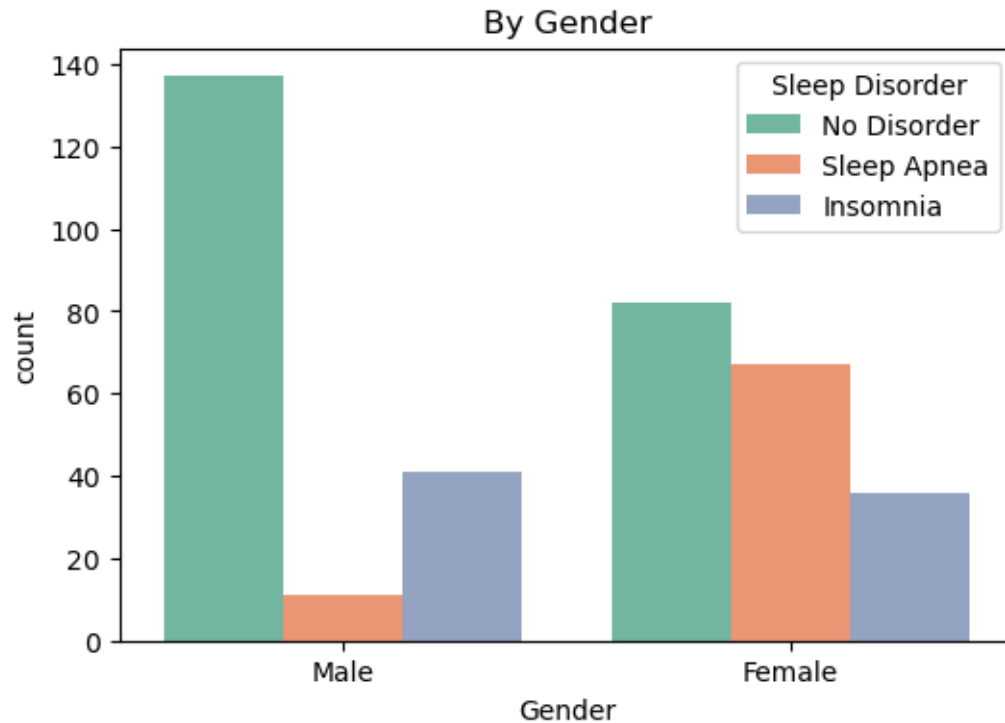
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

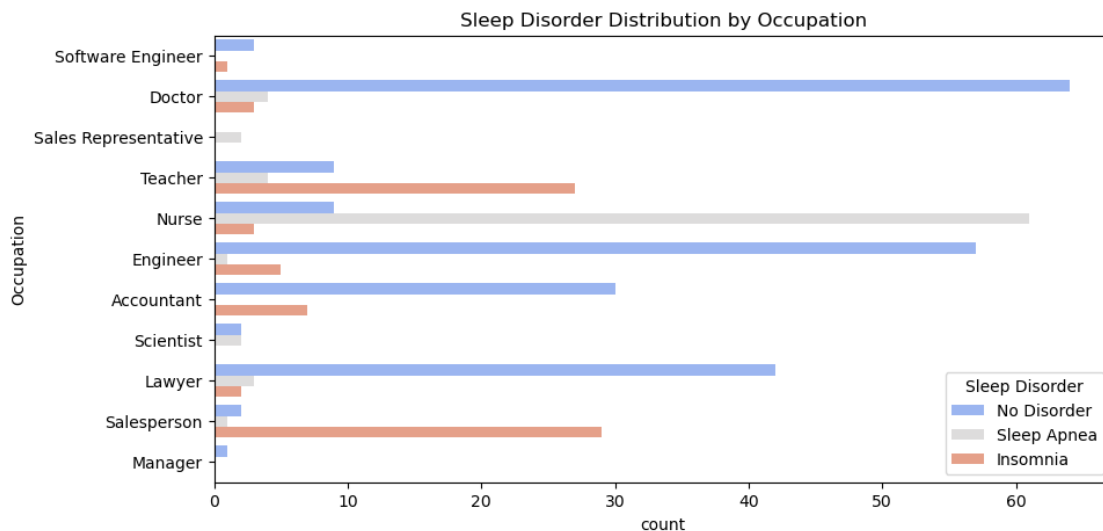
```
sns.countplot(x=df["Sleep Disorder"], palette="coolwarm")
```



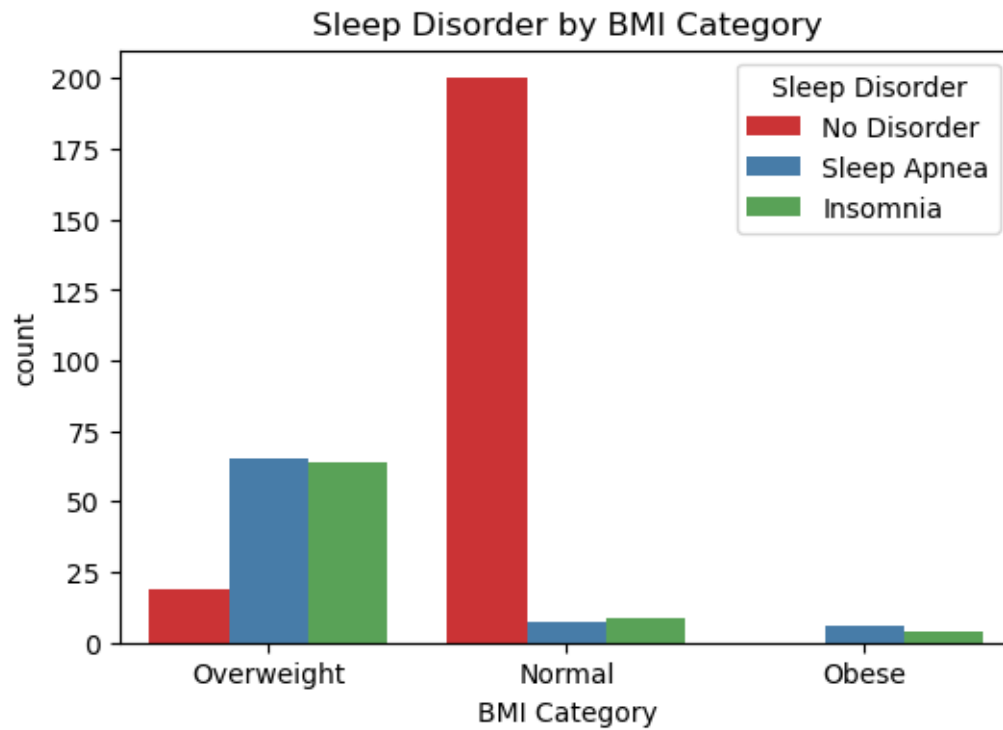
```
[698]: plt.figure(figsize=(6,4))
sns.countplot(x=df["Gender"], hue=df["Sleep Disorder"], palette="Set2")
plt.title("By Gender")
plt.show()
```



```
[699]: plt.figure(figsize=(10,5))
sns.countplot(y="Occupation", hue="Sleep Disorder", data=df, palette="coolwarm")
plt.title("Sleep Disorder Distribution by Occupation")
plt.legend(title="Sleep Disorder")
plt.savefig("DistributionofSleepDisorderOccupation.png", dpi=300,
           bbox_inches='tight')
plt.show()
```

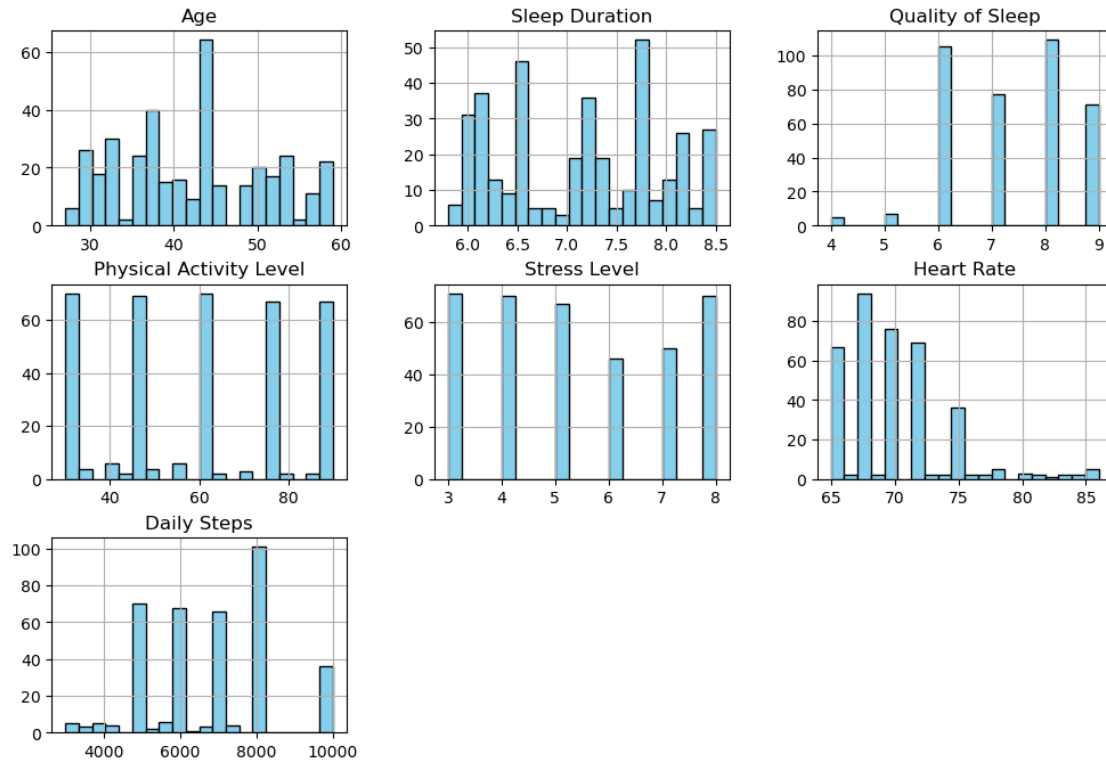


```
[700]: plt.figure(figsize=(6,4))
sns.countplot(x="BMI Category", hue="Sleep Disorder", data=df, palette="Set1")
plt.title("Sleep Disorder by BMI Category")
plt.savefig("DistributionofSleepDisorderBMI.png", dpi=300, bbox_inches='tight')
plt.show()
```

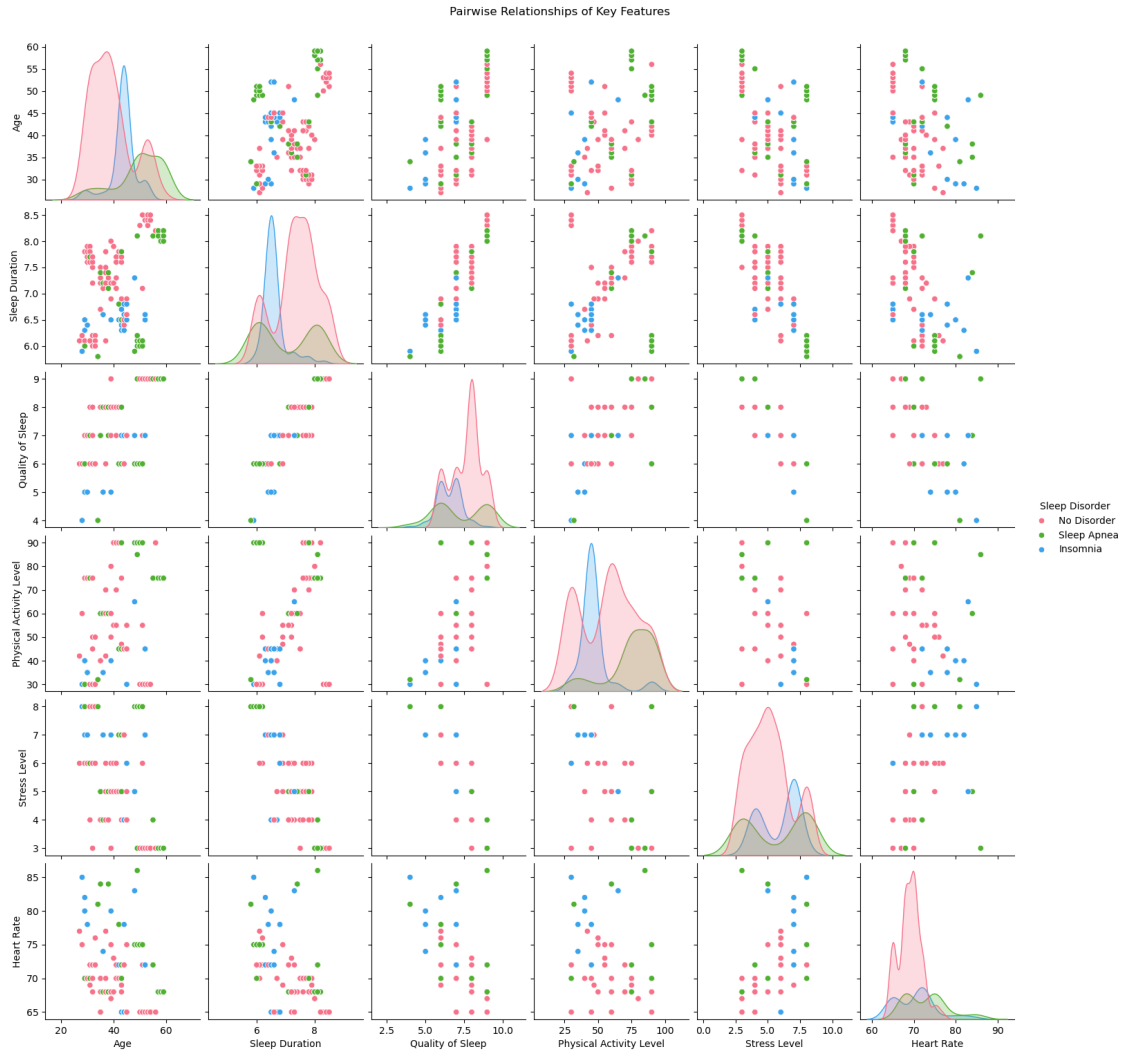


```
[701]: df[num_cols].hist(figsize=(12,8), bins=20, color="skyblue", edgecolor="black")
plt.suptitle("Distribution of Numerical Features")
plt.show()
```

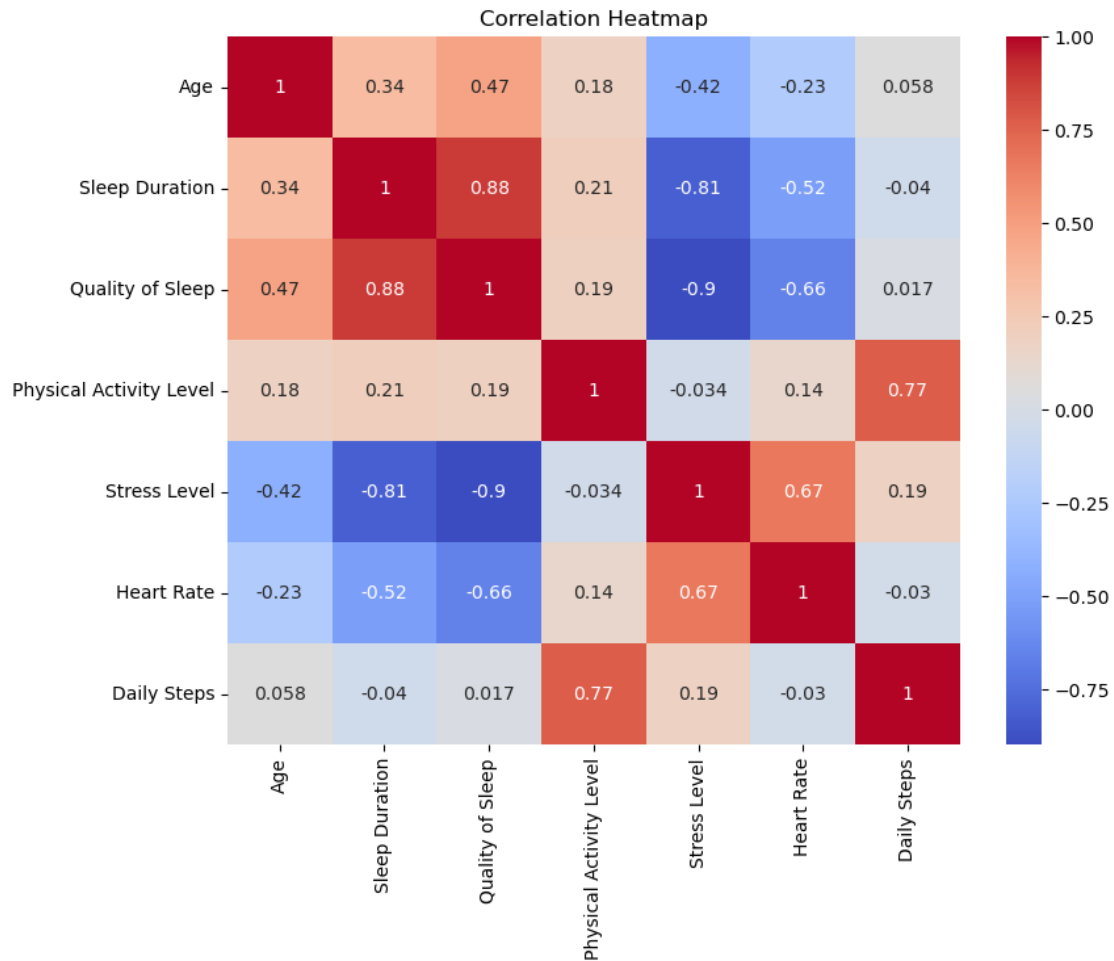
Distribution of Numerical Features



```
[702]: sns.pairplot(df, vars=["Age", "Sleep Duration", "Quality of Sleep",
                             "Physical Activity Level", "Stress Level", "Heart Rate"],
               hue="Sleep Disorder", palette="husl")
plt.suptitle("Pairwise Relationships of Key Features", y=1.02)
plt.savefig("PairwiseRelationshipsofKeyFeatures.png", dpi=300,
           bbox_inches='tight')
plt.show()
```



```
[703]: plt.figure(figsize=(9,7))
sns.heatmap(df[num_cols].corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



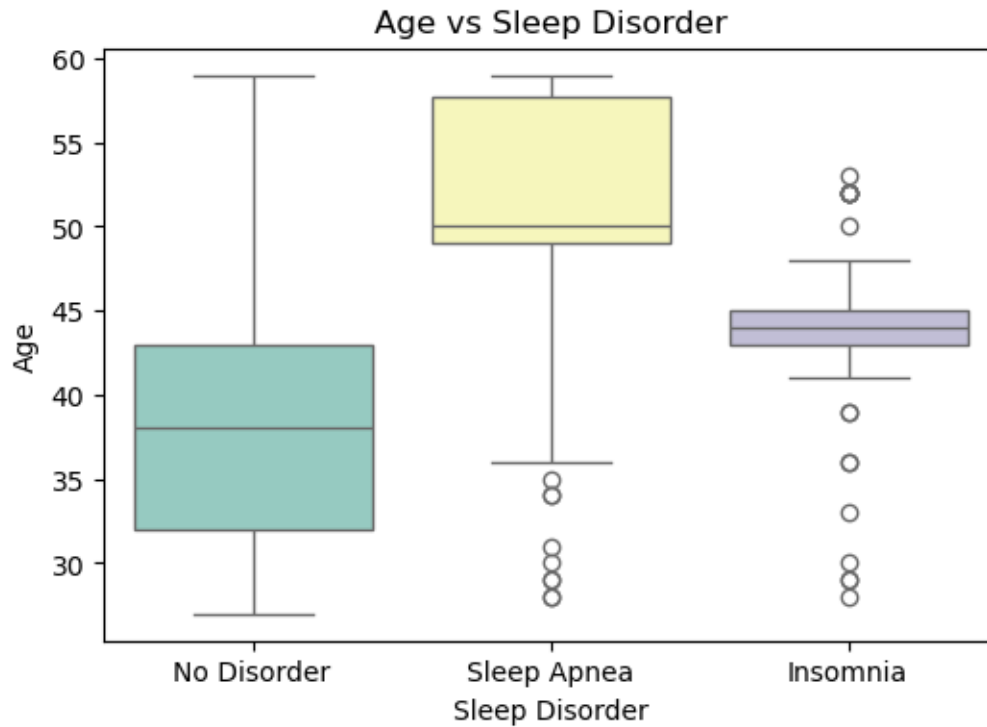
```
[704]: for col in num_cols:
        plt.figure(figsize=(6,4))
        sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
        plt.title(f"{col} vs Sleep Disorder")
        plt.show()
```

C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1004960844.py:3:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

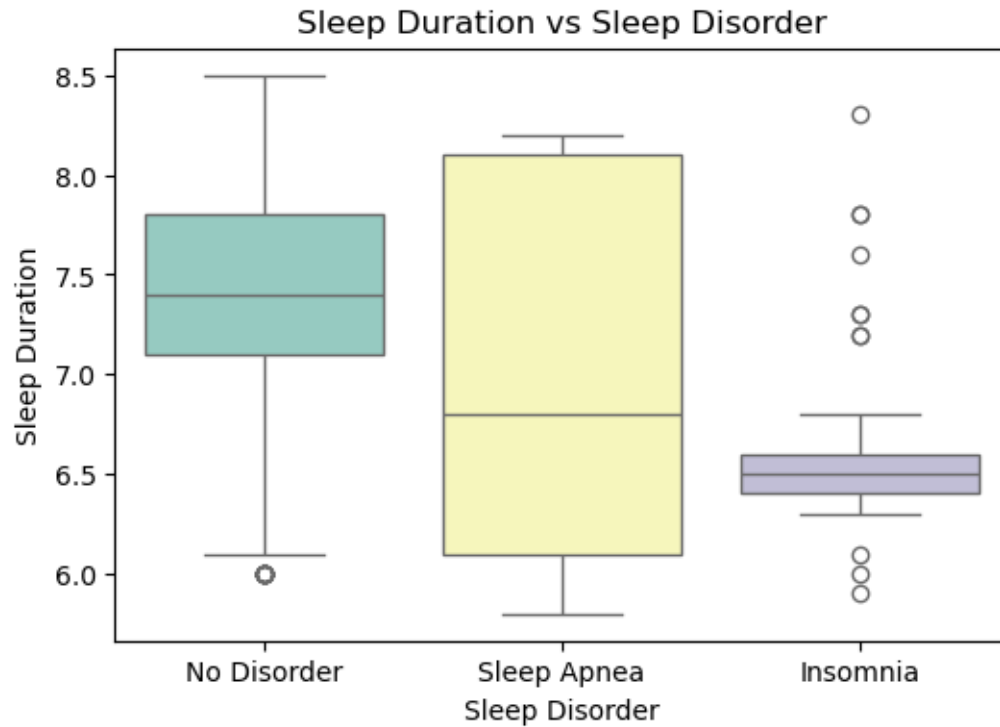
```
sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
```



C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1004960844.py:3:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

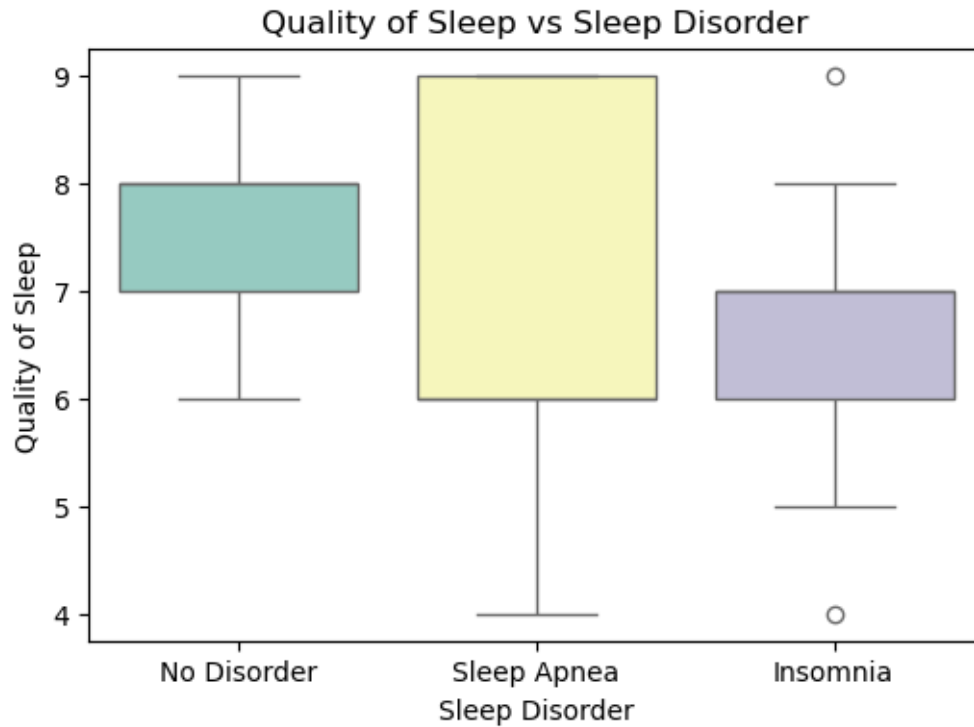
```
sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
```



```
C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1004960844.py:3:  
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

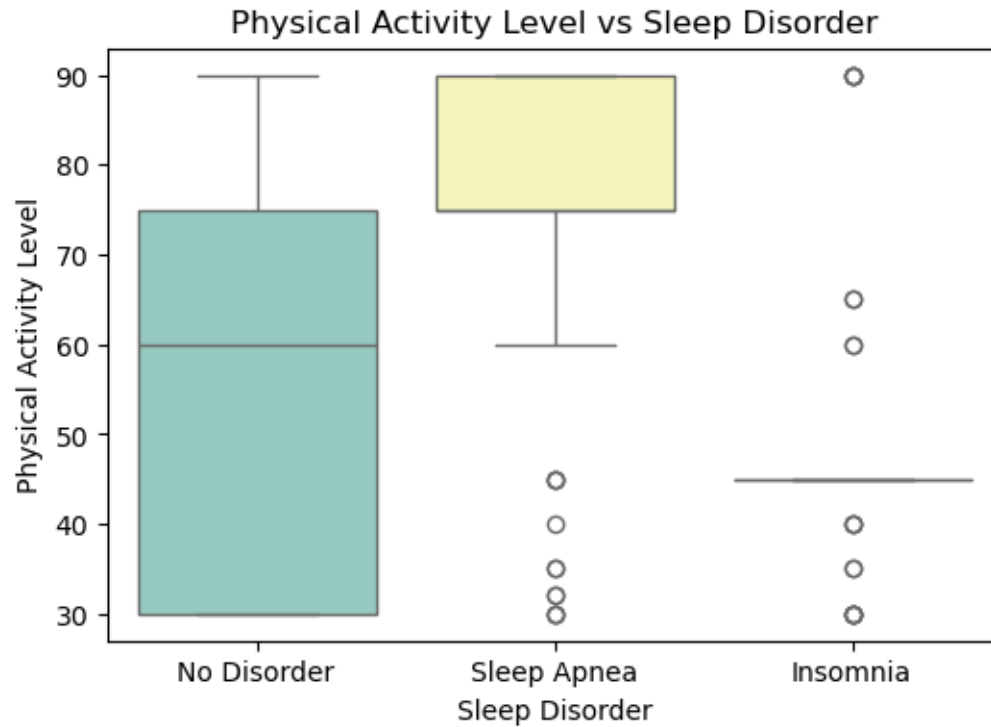
```
sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
```



C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1004960844.py:3:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

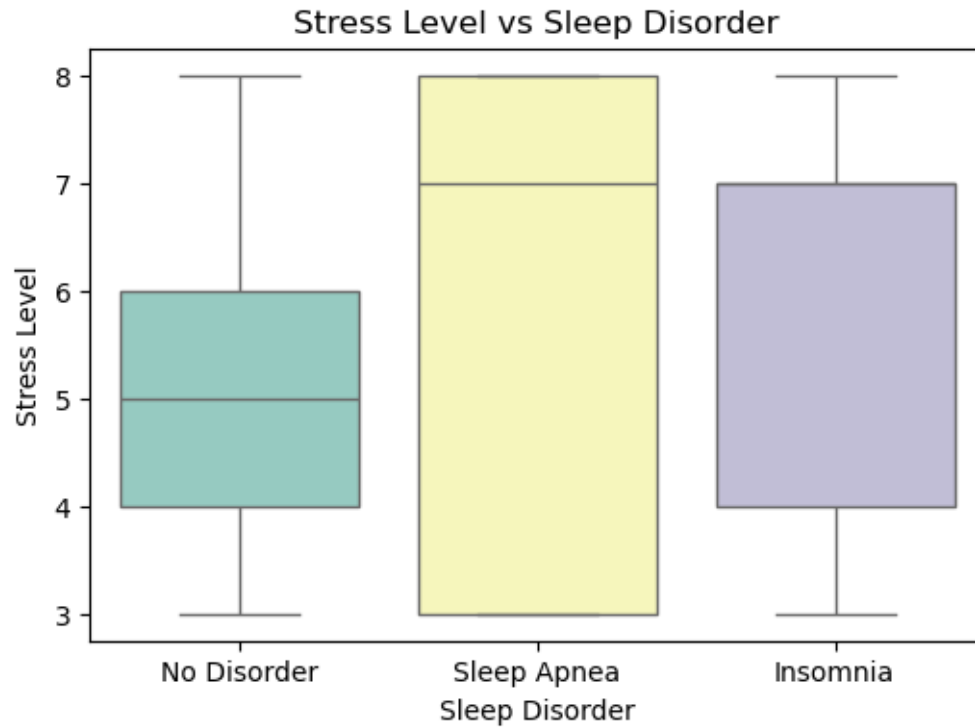
```
sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
```



C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1004960844.py:3:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

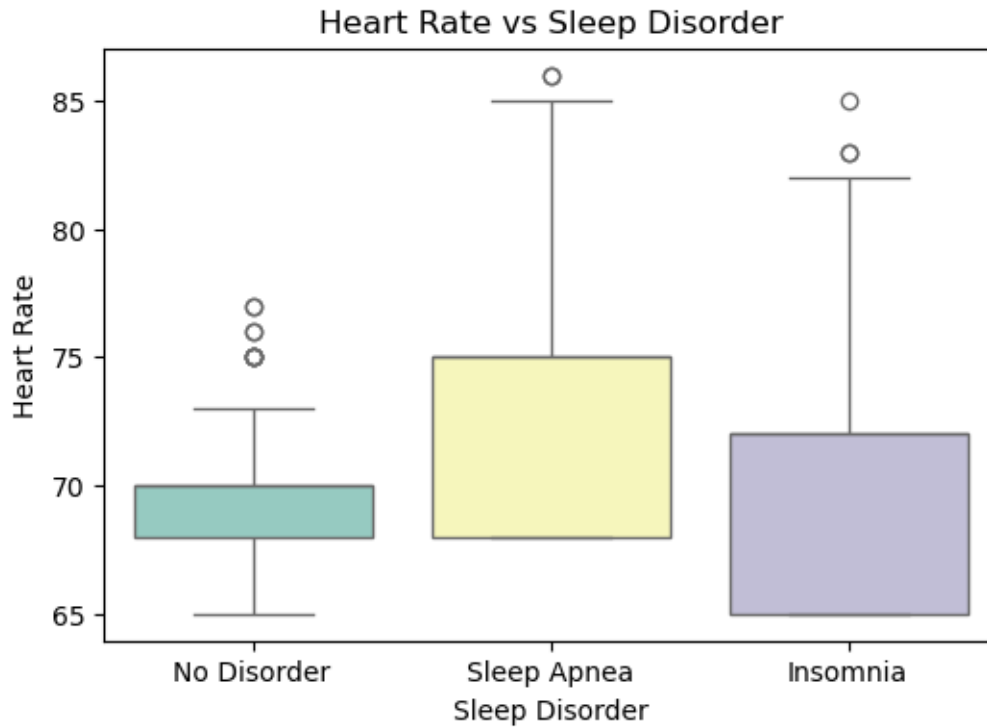
```
sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
```



```
C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1004960844.py:3:  
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

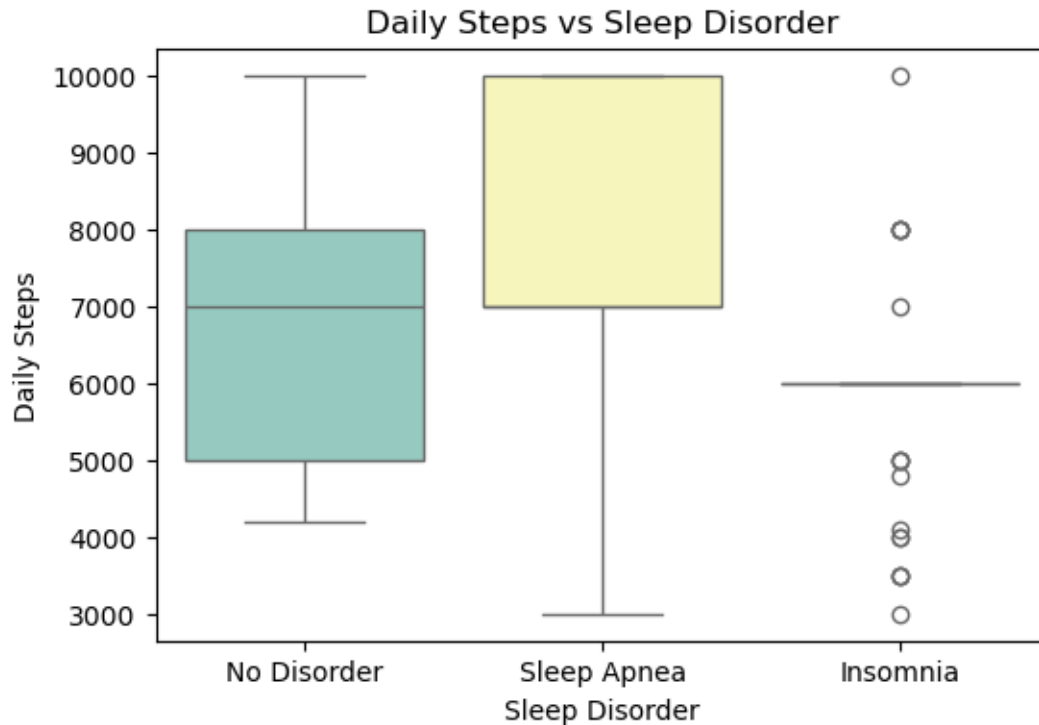
```
sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
```



```
C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1004960844.py:3:  
FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x="Sleep Disorder", y=col, data=df, palette="Set3")
```



```
[705]: def outlier_summary(column):
    q1 = df[column].quantile(0.25)
    q3 = df[column].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    outliers = ((df[column] < lower) | (df[column] > upper)).sum()
    return outliers

print("\n--- Potential Outliers by Column ---")
for col in num_cols:
    print(f"{col}: {outlier_summary(col)} outliers")
```

```
--- Potential Outliers by Column ---
Age: 0 outliers
Sleep Duration: 0 outliers
Quality of Sleep: 0 outliers
Physical Activity Level: 0 outliers
Stress Level: 0 outliers
Heart Rate: 15 outliers
Daily Steps: 0 outliers
```

1.0.5 Data Encoding

```
[707]: cat_cols = df.select_dtypes(include="object").columns.tolist()
label_encoders = {}

for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le
```

1.0.6 Train/Test Data Split

```
[709]: # Feature/target
X = df.drop("Sleep Disorder", axis=1)
y = df["Sleep Disorder"]
```

```
[710]: X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=RANDOM_STATE
)
```

```
[711]: # Standardize features for LR / NB
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

classes_sorted = sorted(y.unique())
sleep_labels = label_encoders["Sleep Disorder"].classes_
```

1.0.7 ROC Plot function

```
[713]: def plot_roc(y_true, y_score, model):
    y_bin = label_binarize(y_true, classes=classes_sorted)
    colors = ["red", "green", "blue", "orange", "purple", "brown"]

    plt.figure(figsize=(7,6))
    for i in range(len(classes_sorted)):
        fpr, tpr, _ = roc_curve(y_bin[:,i], y_score[:,i])
        auc_val = auc(fpr, tpr)
        name = sleep_labels[i]
        plt.plot(fpr, tpr, color=colors[i%len(colors)],
                label=f"{name} (AUC={auc_val:.2f})")

    plt.plot([0,1], [0,1], 'k--')
    plt.title(f"ROC Curve - {model}")
    plt.legend()
```

```
plt.show()
```

1.0.8 Logistic Regression Model

```
[715]: logreg = LogisticRegression(max_iter=1000, class_weight="balanced")
logreg.fit(X_train_scaled, y_train)

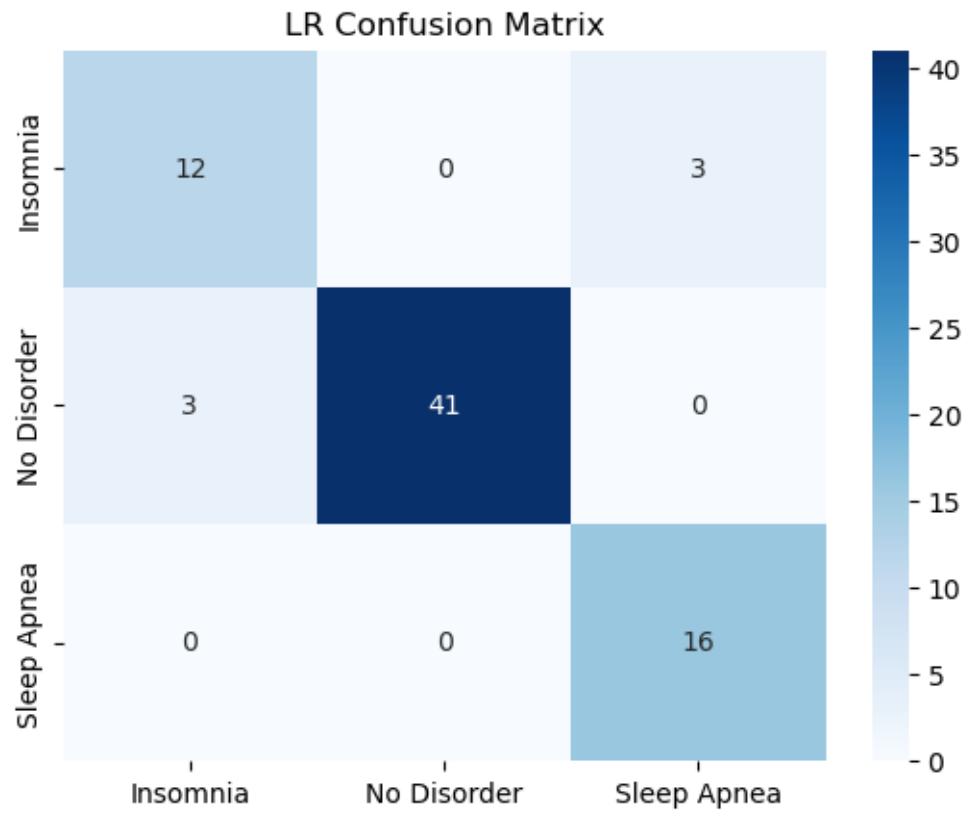
y_pred_lr = logreg.predict(X_test_scaled)
```

```
[716]: print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

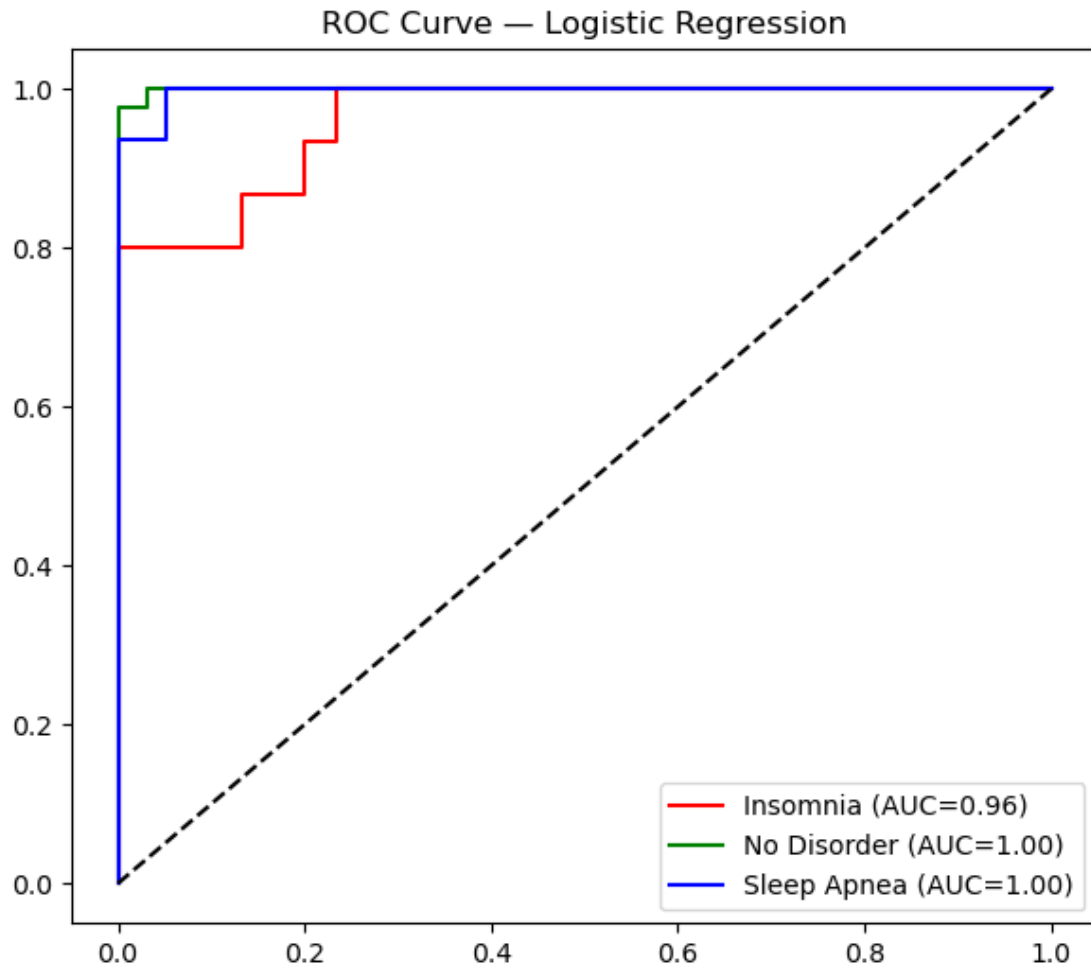
Accuracy: 0.92

	precision	recall	f1-score	support
0	0.80	0.80	0.80	15
1	1.00	0.93	0.96	44
2	0.84	1.00	0.91	16
accuracy			0.92	75
macro avg	0.88	0.91	0.89	75
weighted avg	0.93	0.92	0.92	75

```
[717]: sns.heatmap(confusion_matrix(y_test,y_pred_lr),annot=True,cmap="Blues",
                xticklabels=sleep_labels, yticklabels=sleep_labels)
plt.title("LR Confusion Matrix")
plt.show()
```



```
[718]: plot_roc(y_test, logreg.predict_proba(X_test_scaled), "Logistic Regression")
```



```
[719]: #Feature Influence
coef_abs = np.abs(logreg.coef_).mean(axis=0)    # avg |coef| across classes

fi_lr = pd.DataFrame({
    "Feature": X.columns,
    "Mean Influence": coef_abs
}).sort_values(by="Mean Influence", ascending=False)

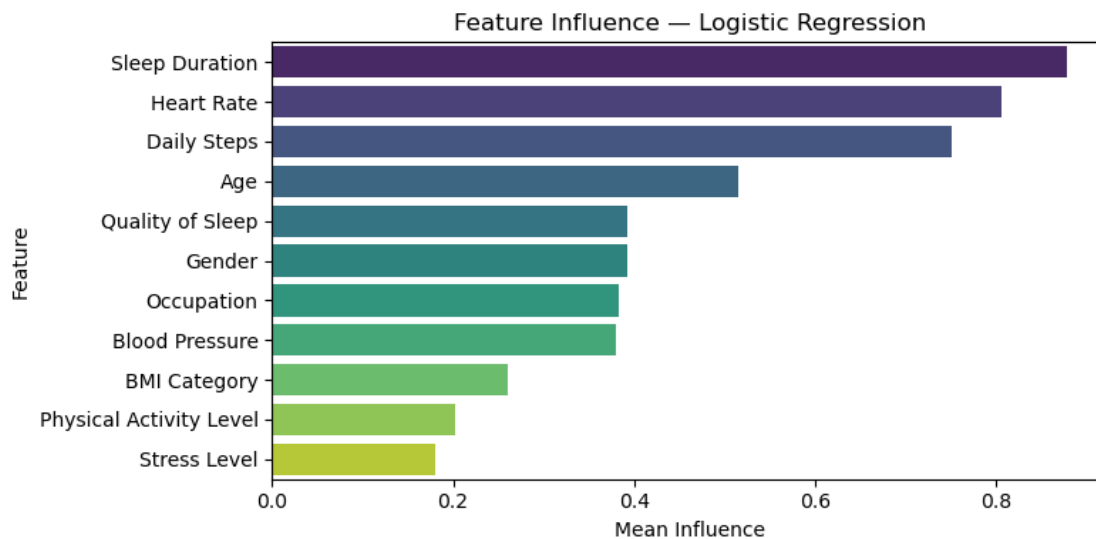
plt.figure(figsize=(8,4))
sns.barplot(data=fi_lr, x="Mean Influence", y="Feature", palette="viridis")
plt.title("Feature Influence - Logistic Regression")
plt.tight_layout()
plt.show()

print("\nTop Features (Logistic Regression):")
print(fi_lr.head(10).to_string(index=False))
```

C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1514008506.py:10:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=fi_lr, x="Mean Influence", y="Feature", palette="viridis")
```



Top Features (Logistic Regression):

Feature	Mean Influence
Sleep Duration	0.878515
Heart Rate	0.805771
Daily Steps	0.751787
Age	0.514731
Quality of Sleep	0.392982
Gender	0.392683
Occupation	0.383130
Blood Pressure	0.379337
BMI Category	0.259709
Physical Activity Level	0.202093

1.0.9 Naive Bayes

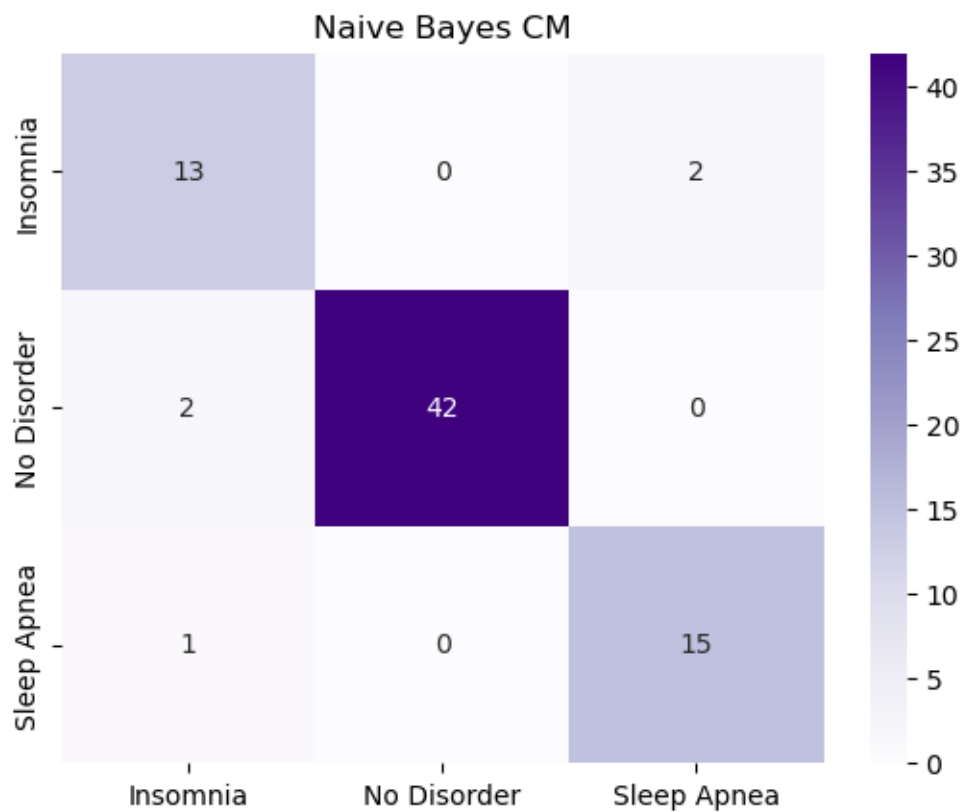
```
[721]: nb = GaussianNB()  
nb.fit(X_train_scaled, y_train)  
  
y_pred_nb = nb.predict(X_test_scaled)
```

```
[722]: print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb))
```

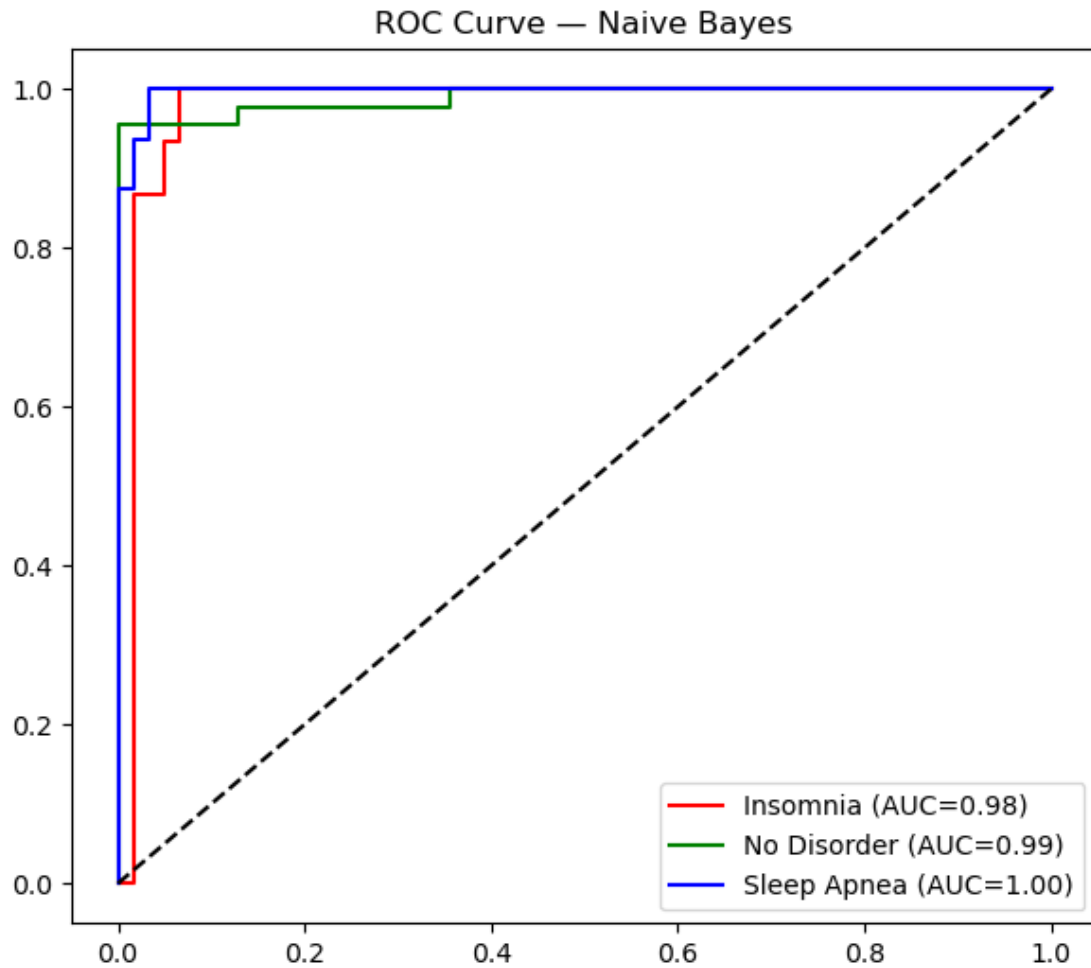
Accuracy: 0.9333333333333333

	precision	recall	f1-score	support
0	0.81	0.87	0.84	15
1	1.00	0.95	0.98	44
2	0.88	0.94	0.91	16
accuracy			0.93	75
macro avg	0.90	0.92	0.91	75
weighted avg	0.94	0.93	0.93	75

```
[723]: sns.heatmap(confusion_matrix(y_test,y_pred_nb),annot=True,cmap="Purples",
                    xticklabels=sleep_labels, yticklabels=sleep_labels)
plt.title("Naive Bayes CM")
plt.show()
```



```
[724]: plot_roc(y_test, nb.predict_proba(X_test_scaled),"Naive Bayes")
```



```
[725]: #Feature Influence
means_nb = np.abs(nb.theta_).mean(axis=0)    # avg |class mean| per feature

fi_nb = pd.DataFrame({
    "Feature": X.columns,
    "Mean Influence": means_nb
}).sort_values(by="Mean Influence", ascending=False)

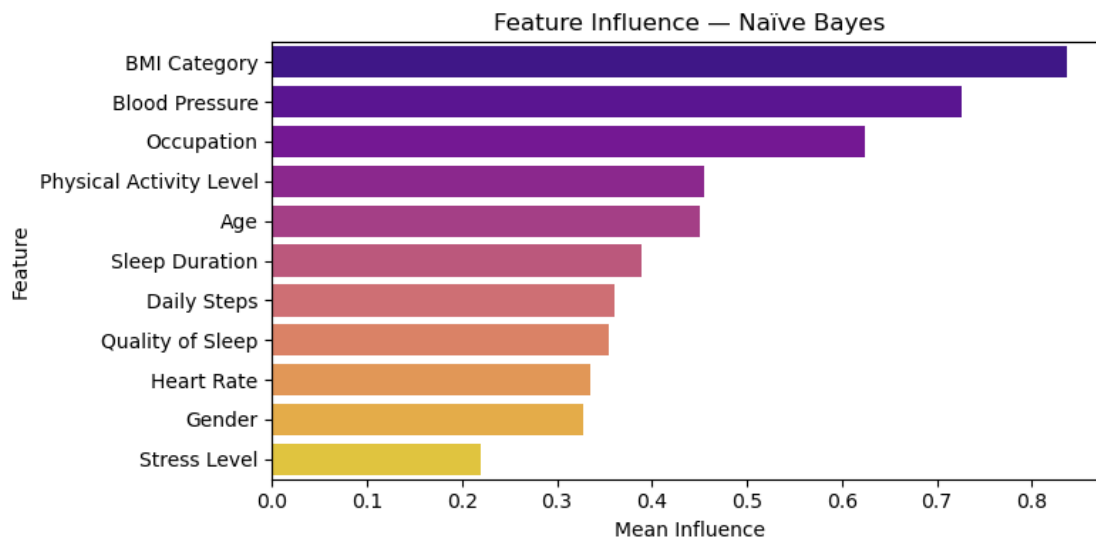
plt.figure(figsize=(8,4))
sns.barplot(data=fi_nb, x="Mean Influence", y="Feature", palette="plasma")
plt.title("Feature Influence - Naïve Bayes")
plt.tight_layout()
plt.show()

print("\nTop Features (Naïve Bayes):")
print(fi_nb.head(10).to_string(index=False))
```

C:\Users\molin\AppData\Local\Temp\ipykernel_39696\4169551485.py:10:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=fi_nb, x="Mean Influence", y="Feature", palette="plasma")
```



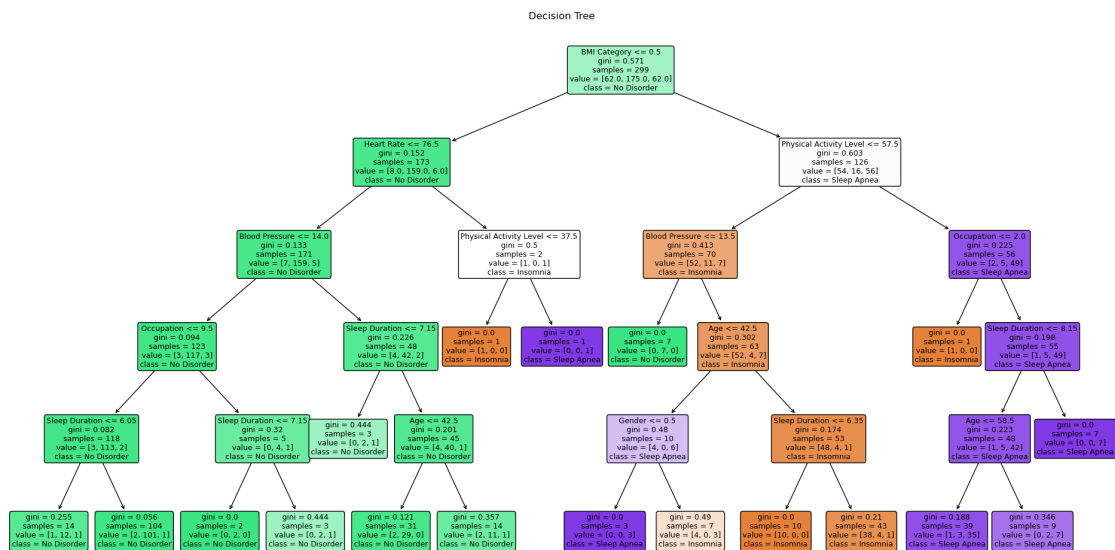
Top Features (Naïve Bayes):

Feature	Mean Influence
BMI Category	0.837174
Blood Pressure	0.726220
Occupation	0.625032
Physical Activity Level	0.454480
Age	0.449998
Sleep Duration	0.389482
Daily Steps	0.359889
Quality of Sleep	0.353963
Heart Rate	0.334653
Gender	0.328104

1.0.10 Decision Tree

```
[727]: dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train,y_train)
y_pred_dt = dt.predict(X_test)
```

```
[728]: plt.figure(figsize=(24, 12))
plot_tree(
    dt,
    feature_names=X.columns,
    class_names=sleep_labels,
    filled=True,
    rounded=True,
    max_depth=None,
    fontsize=9
)
plt.title("Decision Tree")
plt.show()
```

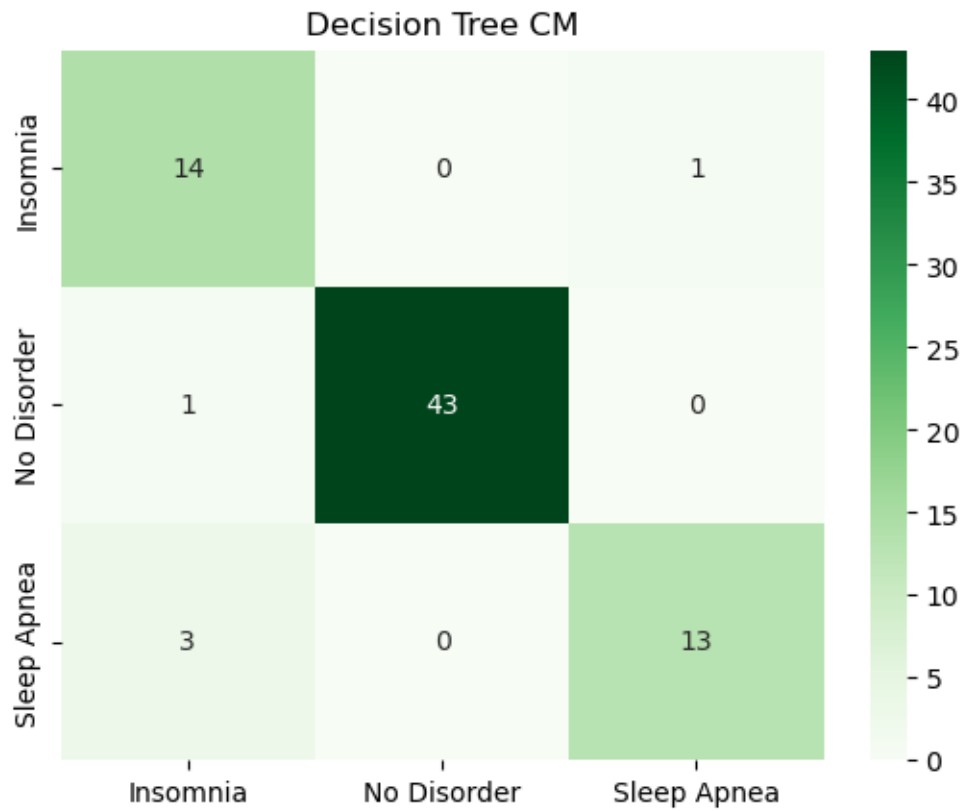


```
[729]: print("Accuracy:", accuracy_score(y_test,y_pred_dt))
print(classification_report(y_test,y_pred_dt))
```

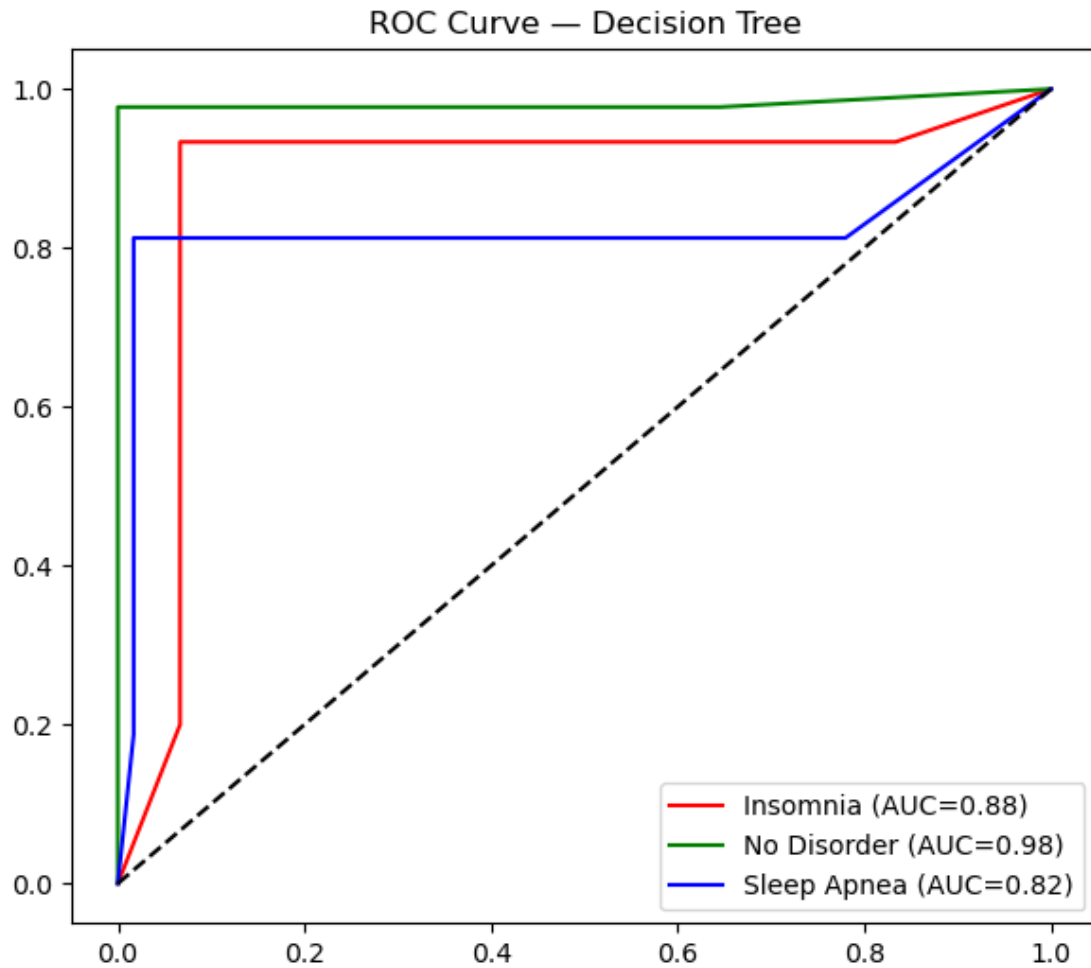
Accuracy: 0.9333333333333333

	precision	recall	f1-score	support
0	0.78	0.93	0.85	15
1	1.00	0.98	0.99	44
2	0.93	0.81	0.87	16
accuracy			0.93	75
macro avg	0.90	0.91	0.90	75
weighted avg	0.94	0.93	0.93	75

```
[730]: sns.heatmap(confusion_matrix(y_test,y_pred_dt),annot=True,cmap="Greens",
                  xticklabels=sleep_labels, yticklabels=sleep_labels)
plt.title("Decision Tree CM")
plt.show()
```



```
[731]: plot_roc(y_test,dt.predict_proba(X_test),"Decision Tree")
```



```
[732]: #Feature Influence

importances_dt = dt.feature_importances_

fi_dt = pd.DataFrame({
    "Feature": X.columns,
    "Mean Influence": importances_dt
}).sort_values(by="Mean Influence", ascending=False)

plt.figure(figsize=(8,4))
sns.barplot(data=fi_dt, x="Mean Influence", y="Feature", palette="magma")
plt.title("Feature Influence - Decision Tree")
plt.tight_layout()
plt.show()

print("\nTop Features (Decision Tree):")
```

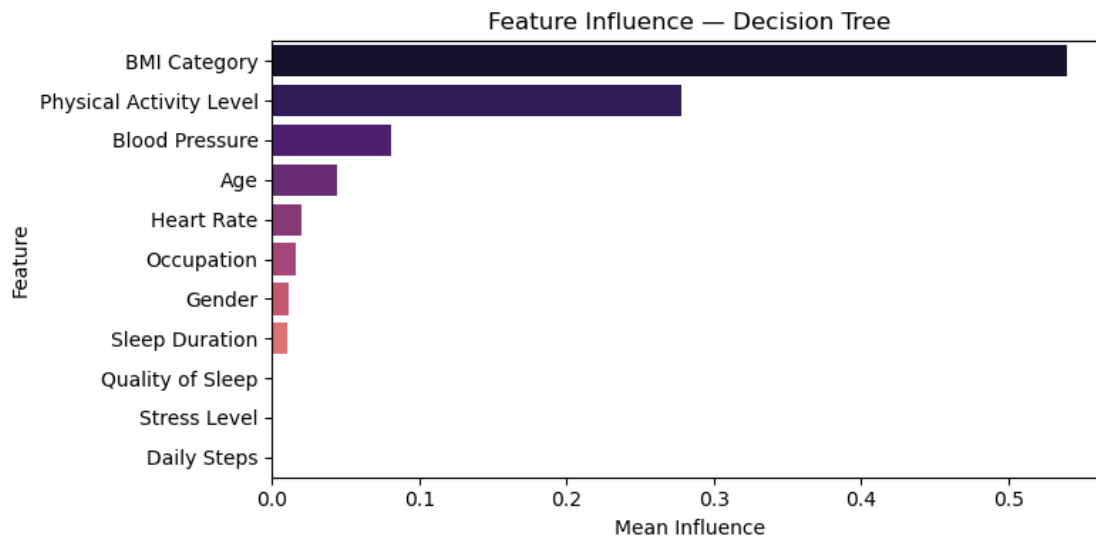
```
print(fi_dt.head(10).to_string(index=False))
```

C:\Users\molin\AppData\Local\Temp\ipykernel_39696\4050472015.py:11:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=fi_dt, x="Mean Influence", y="Feature", palette="magma")
```



Top Features (Decision Tree):

Feature	Mean Influence
BMI Category	0.539936
Physical Activity Level	0.278368
Blood Pressure	0.080444
Age	0.044278
Heart Rate	0.020169
Occupation	0.015870
Gender	0.010788
Sleep Duration	0.010147
Quality of Sleep	0.000000
Stress Level	0.000000

1.0.11 K Nearest Neighbor (KNN)

```
[734]: # Aseguramos que la columna es string
df["Blood Pressure"] = df["Blood Pressure"].astype(str)

# Extraemos solo los números antes y después del "/"
bp = df["Blood Pressure"].str.extract(
    r'(?P<Systolic>\d+)\s*/\s*(?P<Diastolic>\d+)' # captura "123 / 80", "123/
    ↪80", etc.
)

# Convertimos a numérico
bp = bp.apply(pd.to_numeric, errors="coerce")

# Unimos al df original y sacamos la columna vieja
df = df.join(bp).drop(columns=["Blood Pressure"])
```

```
[735]: # Normalizing features
numeric_cols = [
    "Age",
    "Sleep Duration",
    "Quality of Sleep",
    "Physical Activity Level",
    "Stress Level",
    "Heart Rate",
    "Daily Steps",
    "Systolic",
    "Diastolic"
]

# Keep only the columns that exist in your dataset (safe check)
numeric_cols = [col for col in numeric_cols if col in X_train.columns]

scaler = StandardScaler()

# Fit only on training data
scaler.fit(X_train[numeric_cols])

# Transform both train and test sets
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()
X_train_scaled[numeric_cols] = scaler.transform(X_train[numeric_cols])
X_test_scaled[numeric_cols] = scaler.transform(X_test[numeric_cols])

# Checking it out
print("Scaled numeric features:")
print(X_train_scaled[numeric_cols].describe().round(2).T)
```

Scaled numeric features:

	count	mean	std	min	25%	50%	75%	max
Age	299.0	0.0	1.0	-1.78	-0.73	0.09	0.90	1.95
Sleep Duration	299.0	-0.0	1.0	-1.70	-0.87	0.08	0.84	1.72
Quality of Sleep	299.0	0.0	1.0	-2.81	-1.12	-0.27	0.58	1.42
Physical Activity Level	299.0	-0.0	1.0	-1.43	-0.70	0.02	0.74	1.46
Stress Level	299.0	0.0	1.0	-1.36	-0.79	-0.22	0.92	1.49
Heart Rate	299.0	0.0	1.0	-1.26	-0.51	-0.00	0.50	3.78
Daily Steps	299.0	0.0	1.0	-2.40	-0.54	0.08	0.70	1.94

```
[736]: # Identify columns
num_cols = X_train.select_dtypes(include=["number"]).columns
cat_cols = X_train.select_dtypes(exclude=["number"]).columns

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])

# ColumnTransformer (this must be an object, not a tuple/list)
preprocess = ColumnTransformer(
    transformers=[
        ("num", num_pipe, num_cols),
        ("cat", cat_pipe, cat_cols),
    ],
    remainder="drop"
)

pipe = Pipeline([
    ("prep", preprocess),
    ("knn", KNeighborsClassifier(metric="euclidean"))
])

# Hyperparameter Grid
k_neighbors = list(range(1, 21, 2)) # 1,3,5,...,19
param_grid = {
    "knn__n_neighbors": k_neighbors,
    "knn__weights": ["uniform", "distance"],
}

# 10 fold stratified CV
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
```

```

# Grid search
gs = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv=cv,
    scoring="accuracy",
)

gs.fit(X_train, y_train)

best_k = gs.best_params_["knn__n_neighbors"]
print("Best k:", best_k)
print("Best params:", gs.best_params_)
print("Best CV accuracy:", round(gs.best_score_, 4))
print("Test accuracy with best model:", round(gs.score(X_test, y_test), 4))

```

Best k: 5
Best params: {'knn__n_neighbors': 5, 'knn__weights': 'uniform'}
Best CV accuracy: 0.8728
Test accuracy with best model: 0.9467

```

[737]: # Evaluating
# CV accuracy
print("Best CV accuracy:", round(gs.best_score_, 4))
print("Test accuracy with best model:", round(gs.score(X_test, y_test), 4))

# Finding y_pred
best_model = gs.best_estimator_
y_pred = best_model.predict(X_test)

# Confusion Matrix
print("\nConfusion matrix:\n", confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification report:\n", classification_report(y_test, y_pred,
    digits=3))

```

Best CV accuracy: 0.8728
Test accuracy with best model: 0.9467

Confusion matrix:

```

[[13  1  1]
 [ 1 43  0]
 [ 0  1 15]]

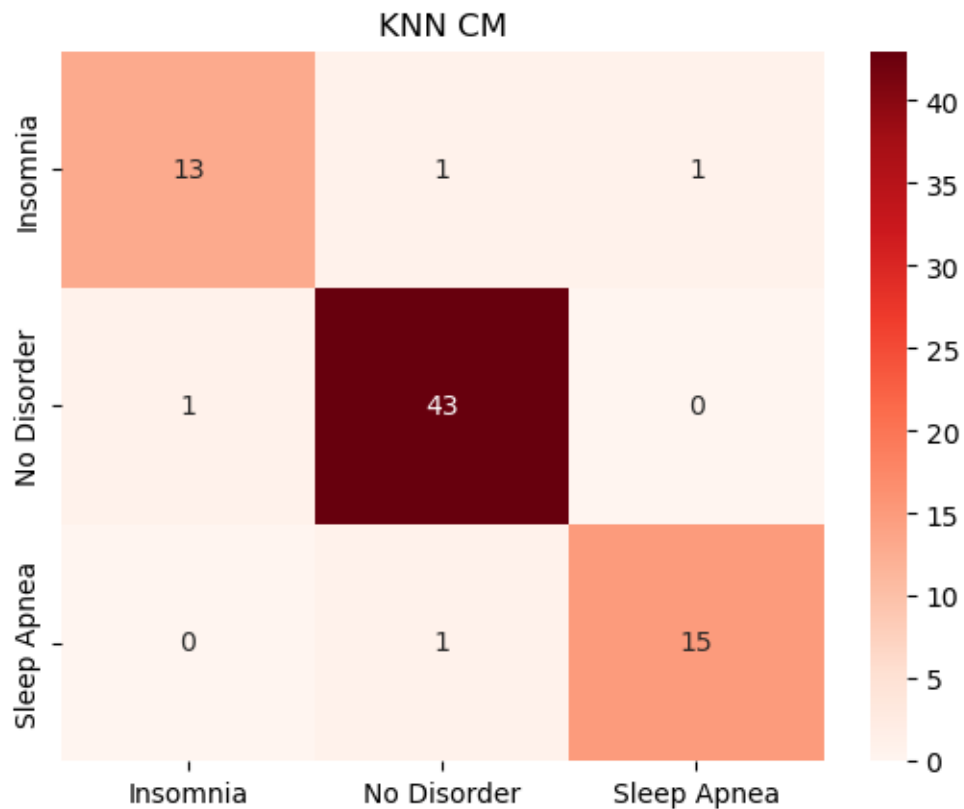
```

Classification report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.929	0.867	0.897	15
1	0.956	0.977	0.966	44
2	0.938	0.938	0.938	16
accuracy			0.947	75
macro avg	0.941	0.927	0.933	75
weighted avg	0.946	0.947	0.946	75

```
[738]: sns.heatmap(confusion_matrix(y_test,y_pred),annot=True,cmap="Reds",
                  xticklabels=sleep_labels, yticklabels=sleep_labels)
plt.title("KNN CM")
plt.show()
```



```
[739]: best_model = gs.best_estimator_

y_score = best_model.predict_proba(X_test)

# Checking if model is not overfitting
print("Train accuracy:", best_model.score(X_train, y_train))
```

```

print("Test accuracy:", best_model.score(X_test, y_test))

# --- Binarize the target (convert classes -> 0/1 columns)
classes = np.unique(y_test)
y_test_bin = label_binarize(y_test, classes=classes)

# --- Compute ROC curve and AUC for each class
fpr, tpr, roc_auc = {}, {}, {}
for i, cls in enumerate(classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# --- Compute macro-average AUC
roc_auc["macro"] = roc_auc_score(y_test_bin, y_score, average="macro")

# --- Print numeric results
for i, cls in enumerate(classes):
    print(f"AUC for class '{cls}': {roc_auc[i]:.3f}")
print(f"\nMacro-average AUC: {roc_auc['macro']:.3f}")

# --- Plot ROC curves
plt.figure(figsize=(7, 6))
for i, cls in enumerate(classes):
    plt.plot(fpr[i], tpr[i], lw=2,
             label=f"{cls} (AUC = {roc_auc[i]:.2f})")
plt.plot([0, 1], [0, 1], "k--", lw=1)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves - Sleep Disorder Classification")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()

```

Train accuracy: 0.8795986622073578

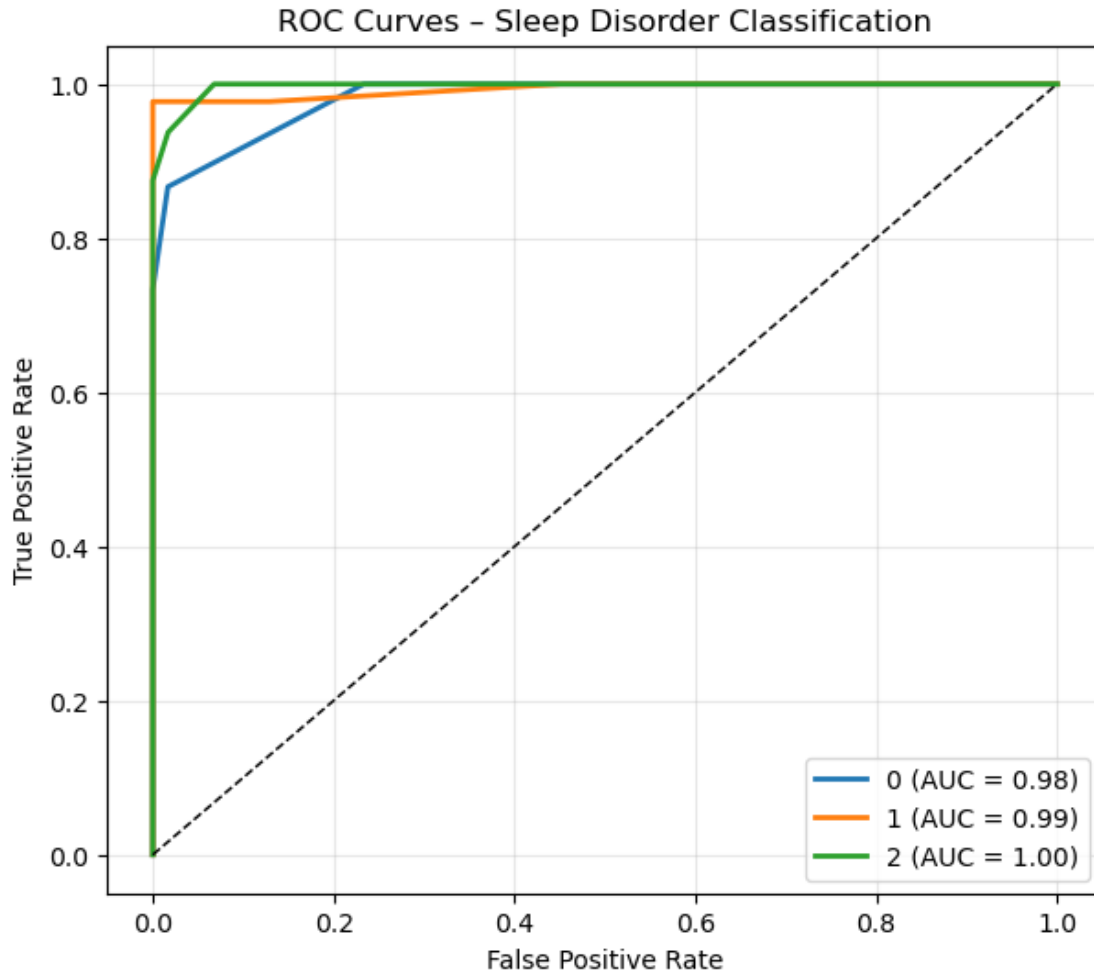
Test accuracy: 0.9466666666666667

AUC for class '0': 0.982

AUC for class '1': 0.993

AUC for class '2': 0.997

Macro-average AUC: 0.991



1.0.12 K Means Clustering

```
[741]: # remove index variable
df_clust.drop(columns=['Person ID'], inplace=True)

# change NaN for No Disorder
df_clust['Sleep Disorder'].fillna('No Disorder', inplace=True)

# create two new variable to preserve the nature of the original variable,
# numerically.
df_clust[['Systolic_BP', 'Diastolic_BP']] = df_clust['Blood Pressure'].str.
    .split('/', expand=True).astype(int)
df_clust.drop(columns=['Blood Pressure'], inplace=True)

df_clust['BMI Category'] = df_clust['BMI Category'].replace({
    'Normal Weight': 'Normal'
})
```

```

})
print(df_clust['BMI Category'].unique())

df_clust['Gender'] = df_clust['Gender'].map({'Male': 0, 'Female': 1})

# Occupation
df_clust = pd.get_dummies(df_clust, columns=['Occupation'], prefix='Occ')

# Sleep Disorder
df_clust = pd.get_dummies(df_clust, columns=['Sleep Disorder'],
    ↪prefix='SleepDisorder')

# BMI Category
bmi_order = {
    'Normal': 0,
    'Overweight': 1,
    'Obese': 2
}

df_clust['BMI Category'] = df_clust['BMI Category'].map(bmi_order)

df_clust.info

```

```
['Overweight' 'Normal' 'Obese']
```

C:\Users\molin\AppData\Local\Temp\ipykernel_39696\1688102609.py:5:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_clust['Sleep Disorder'].fillna('No Disorder', inplace=True)
```

```
[741]: <bound method DataFrame.info of          Gender  Age  Sleep Duration  Quality of
Sleep  Physical Activity Level \
0          0   27          6.1          6          42
1          0   28          6.2          6          60
2          0   28          6.2          6          60
3          0   28          5.9          4          30
4          0   28          5.9          4          30
..      ...  ...
369      1   59          8.1          9          75
```

370	1	59	8.0	9	75
371	1	59	8.1	9	75
372	1	59	8.1	9	75
373	1	59	8.1	9	75

	Stress Level	BMI Category	Heart Rate	Daily Steps	Systolic_BP	...	\
0	6	1	77	4200	126	...	
1	8	0	75	10000	125	...	
2	8	0	75	10000	125	...	
3	8	2	85	3000	140	...	
4	8	2	85	3000	140	...	
..	...	...	...	...	...	...	
369	3	1	68	7000	140	...	
370	3	1	68	7000	140	...	
371	3	1	68	7000	140	...	
372	3	1	68	7000	140	...	
373	3	1	68	7000	140	...	

	Occ_Manager	Occ_Nurse	Occ_Sales Representative	Occ_Salesperson	\
0	False	False	False	False	
1	False	False	False	False	
2	False	False	False	False	
3	False	False	True	False	
4	False	False	True	False	
..	...	...	...	...	
369	False	True	False	False	
370	False	True	False	False	
371	False	True	False	False	
372	False	True	False	False	
373	False	True	False	False	

	Occ_Scientist	Occ_Software Engineer	Occ_Teacher	\
0	False	True	False	
1	False	False	False	
2	False	False	False	
3	False	False	False	
4	False	False	False	
..	...	...	...	
369	False	False	False	
370	False	False	False	
371	False	False	False	
372	False	False	False	
373	False	False	False	

	SleepDisorder_Insomnia	SleepDisorder_No Disorder	\
0	False	True	
1	False	True	

2	False	True
3	False	False
4	False	False
..	...	...
369	False	False
370	False	False
371	False	False
372	False	False
373	False	False

	SleepDisorder_Sleep Apnea
0	False
1	False
2	False
3	True
4	True
..	...
369	True
370	True
371	True
372	True
373	True

[374 rows x 25 columns]>

```
[742]: df_clust[['Systolic_BP', 'Diastolic_BP']] = df_clust[['Systolic_BP',
↪ 'Diastolic_BP']].astype(int)
```

```
[743]: # double check after encoding
df.select_dtypes(include=['object']).columns
```

```
[743]: Index([], dtype='object')
```

```
[744]: # Scale
scaler_km = StandardScaler()
X_km = scaler_km.fit_transform(df_clust)
```

```
[745]: df_scaled_clust = pd.DataFrame(X_km, columns=df_clust.columns)
# Check means and standard deviations
print(df_scaled_clust.mean().round(2))
print(df_scaled_clust.std().round(2)) # Succesful
```

Gender	0.0
Age	-0.0
Sleep Duration	0.0
Quality of Sleep	0.0
Physical Activity Level	-0.0
Stress Level	0.0

BMI Category	0.0
Heart Rate	-0.0
Daily Steps	-0.0
Systolic_BP	-0.0
Diastolic_BP	-0.0
Occ_Accountant	0.0
Occ_Doctor	-0.0
Occ_Engineer	0.0
Occ_Lawyer	-0.0
Occ_Manager	-0.0
Occ_Nurse	0.0
Occ_Sales Representative	0.0
Occ_Salesperson	0.0
Occ_Scientist	-0.0
Occ_Software Engineer	-0.0
Occ_Teacher	0.0
SleepDisorder_Insomnia	-0.0
SleepDisorder_No Disorder	-0.0
SleepDisorder_Sleep Apnea	0.0
dtype: float64	
Gender	1.0
Age	1.0
Sleep Duration	1.0
Quality of Sleep	1.0
Physical Activity Level	1.0
Stress Level	1.0
BMI Category	1.0
Heart Rate	1.0
Daily Steps	1.0
Systolic_BP	1.0
Diastolic_BP	1.0
Occ_Accountant	1.0
Occ_Doctor	1.0
Occ_Engineer	1.0
Occ_Lawyer	1.0
Occ_Manager	1.0
Occ_Nurse	1.0
Occ_Sales Representative	1.0
Occ_Salesperson	1.0
Occ_Scientist	1.0
Occ_Software Engineer	1.0
Occ_Teacher	1.0
SleepDisorder_Insomnia	1.0
SleepDisorder_No Disorder	1.0
SleepDisorder_Sleep Apnea	1.0
dtype: float64	

```
[746]: z_scores = np.abs(df_scaled_clust)
outlier_fraction = (z_scores > 3).sum() / len(df_scaled_clust)
print("Fraction of data beyond 3 stds:", outlier_fraction.mean())
```

Fraction of data beyond 3 stds: 0.009518716577540108

```
[747]: z_scores = np.abs((X_km - X_km.mean(axis=0)) / X_km.std(axis=0))
outlier_mask = (z_scores > 3).any(axis=1)
print(f"Number of outliers: {outlier_mask.sum()}")

outliers_df = df_clust[outlier_mask] # look into original values
print(outliers_df) # visualize outliers
```

Number of outliers: 86

	Gender	Age	Sleep Duration	Quality of Sleep	Physical Activity Level	\
0	0	27	6.1	6	42	
3	0	28	5.9	4	30	
4	0	28	5.9	4	30	
5	0	28	5.9	4	30	
66	0	32	7.2	8	50	
..	...	...	...	...	...	
307	1	52	6.5	7	45	
308	1	52	6.6	7	45	
309	1	52	6.6	7	45	
310	1	52	6.6	7	45	
311	1	52	6.6	7	45	

	Stress Level	BMI Category	Heart Rate	Daily Steps	Systolic_BP	...	\
0	6	1	77	4200	126	...	
3	8	2	85	3000	140	...	
4	8	2	85	3000	140	...	
5	8	2	85	3000	140	...	
66	6	0	68	7000	118	...	
..	...	...	...	...	...	...	
307	7	1	72	6000	130	...	
308	7	1	72	6000	130	...	
309	7	1	72	6000	130	...	
310	7	1	72	6000	130	...	
311	7	1	72	6000	130	...	

	Occ_Manager	Occ_Nurse	Occ_Sales Representative	Occ_Salesperson	\
0	False	False	False	False	
3	False	False	True	False	
4	False	False	True	False	
5	False	False	False	False	
66	False	False	False	False	
..	...	...	...	...	
307	False	False	False	False	

308	False	False	False	False
309	False	False	False	False
310	False	False	False	False
311	False	False	False	False

	Occ_Scientist	Occ_Software Engineer	Occ_Teacher	\
0	False	True	False	
3	False	False	False	
4	False	False	False	
5	False	True	False	
66	False	False	False	
..	...	...	...	
307	False	False	False	
308	False	False	False	
309	False	False	False	
310	False	False	False	
311	False	False	False	

	SleepDisorder_Insomnia	SleepDisorder_No Disorder	\
0	False	True	
3	False	False	
4	False	False	
5	True	False	
66	False	True	
..	...	...	
307	True	False	
308	True	False	
309	True	False	
310	True	False	
311	True	False	

	SleepDisorder_Sleep Apnea
0	False
3	True
4	True
5	False
66	False
..	...
307	False
308	False
309	False
310	False
311	False

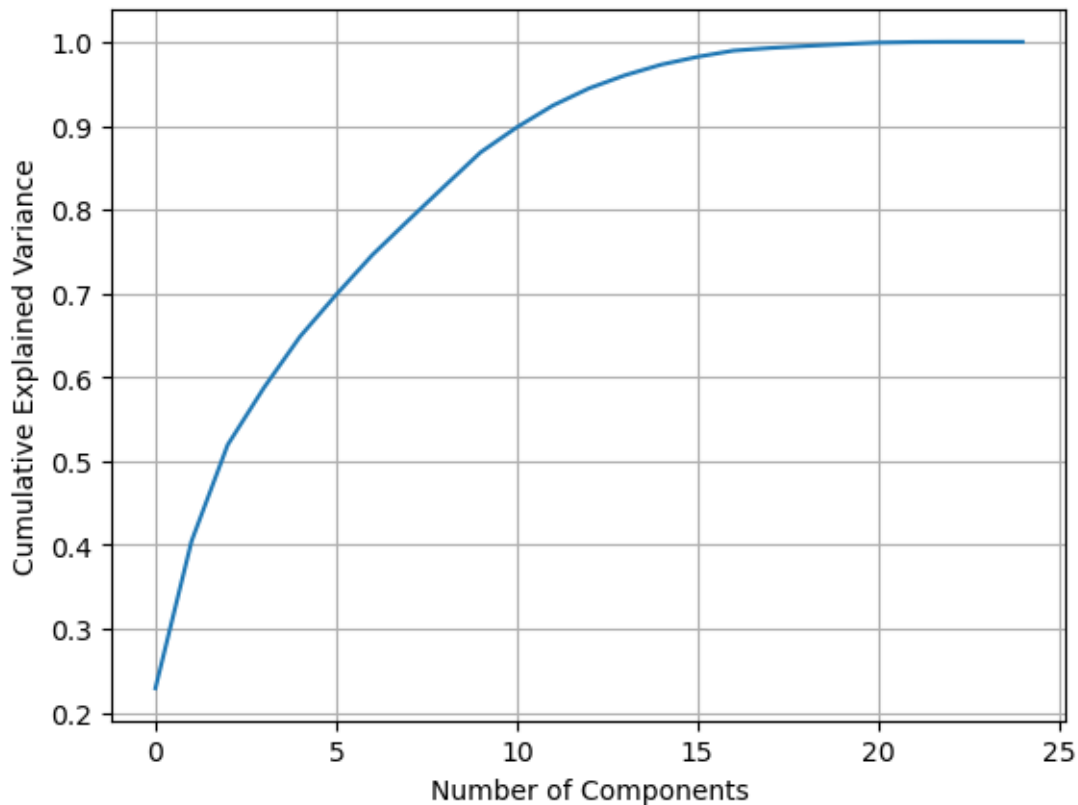
[86 rows x 25 columns]

```
[748]: df_clust.shape
```

[748]: (374, 25)

```
[749]: # PCA
pca = PCA().fit(X_km)

plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.grid(True)
plt.show()
```



```
[750]: pca = PCA(n_components=0.90, random_state=42)
X_pca = pca.fit_transform(X_km)

print("Explained variance per component:", pca.explained_variance_ratio_)
print("Total explained variance:", sum(pca.explained_variance_ratio_))
```

Explained variance per component: [0.22911805 0.17540641 0.11486129 0.06791003
0.06105001 0.04999635
0.04735473 0.04137926 0.04080369 0.04039648 0.02979606 0.02601213]
Total explained variance: 0.9240844940690566

```
[751]: # Record the Total Intra-Cluster Variation
ticv = []

for k in range(1,11):
    kmeans = KMeans(n_clusters=k, n_init=20, random_state=42)

    # Fit kmeans algorithm to the X dataset
    kmeans.fit(X_pca)

    # Record the Total Intra-Cluster Variation for K=k
    ticv.append(kmeans.inertia_)

# Plot the elbow graph
plt.plot(range(1,11),ticv)
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Total Intra-Cluster Variation')
plt.show()
```

C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
 UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
 there are less chunks than available threads. You can avoid it by setting the
 environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
 UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
 there are less chunks than available threads. You can avoid it by setting the
 environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
 UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
 there are less chunks than available threads. You can avoid it by setting the
 environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
 UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
 there are less chunks than available threads. You can avoid it by setting the
 environment variable OMP_NUM_THREADS=2.

warnings.warn(

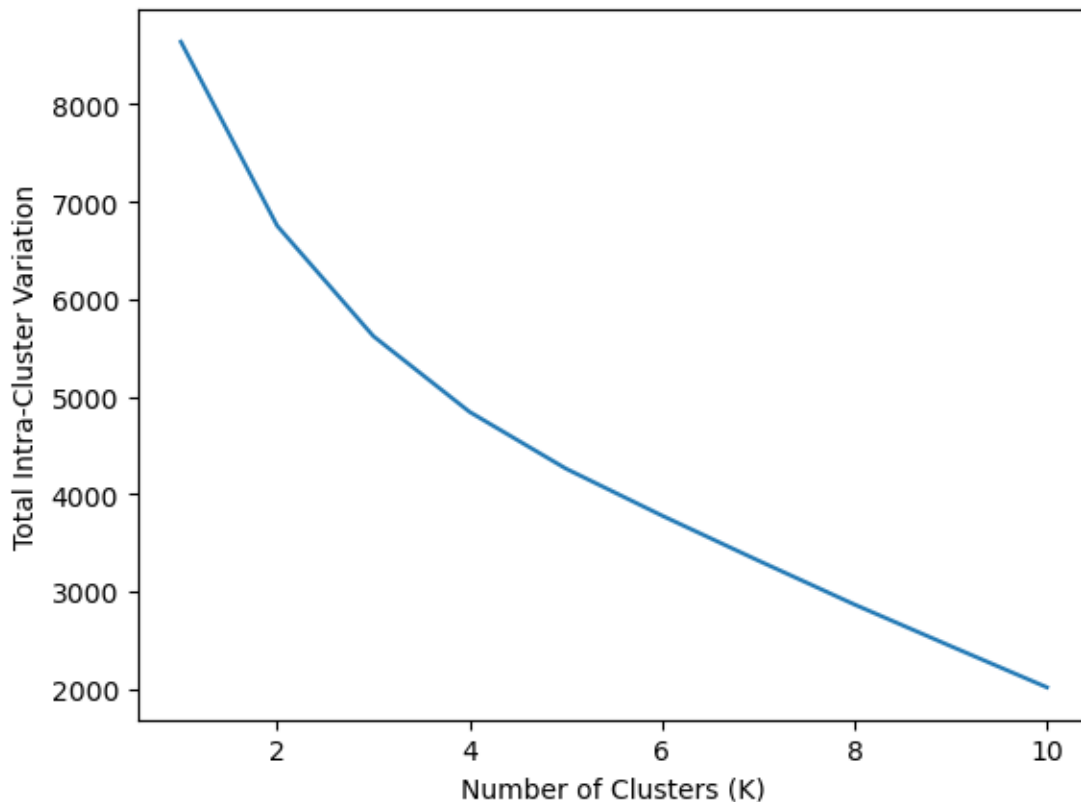
C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
 UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
 there are less chunks than available threads. You can avoid it by setting the
 environment variable OMP_NUM_THREADS=2.

warnings.warn(

C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
 UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
 there are less chunks than available threads. You can avoid it by setting the
 environment variable OMP_NUM_THREADS=2.

```
warnings.warn(  
C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1446:  
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when  
there are less chunks than available threads. You can avoid it by setting the  
environment variable OMP_NUM_THREADS=2.  
warnings.warn(  
C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1446:  
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when  
there are less chunks than available threads. You can avoid it by setting the  
environment variable OMP_NUM_THREADS=2.  
warnings.warn(  
C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1446:  
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when  
there are less chunks than available threads. You can avoid it by setting the  
environment variable OMP_NUM_THREADS=2.  
warnings.warn(  
C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1446:  
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when  
there are less chunks than available threads. You can avoid it by setting the  
environment variable OMP_NUM_THREADS=2.  
warnings.warn(  

```



```
[752]: kmeans = KMeans(n_clusters=4, n_init=20, random_state=42)

kmeans.fit(X_pca)

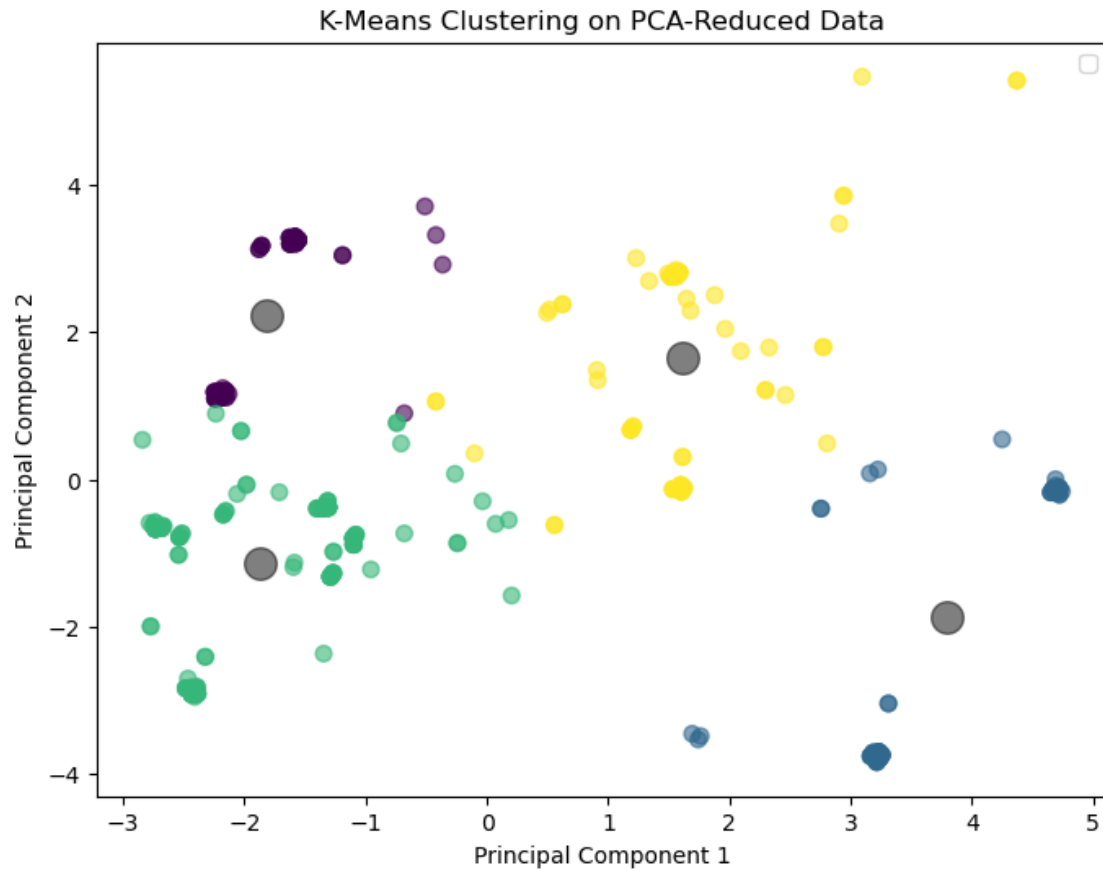
# Get predicted cluster label for each point
y_kmeans = kmeans.predict(X_pca)

# Get the centers of the predicted clusters
centers = kmeans.cluster_centers_

# Plot the clustering result
plt.figure(figsize=(8,6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans, cmap='viridis', s=50, alpha=0.
↪6)
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5)
plt.title("K-Means Clustering on PCA-Reduced Data")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()
```

C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
 UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
 there are less chunks than available threads. You can avoid it by setting the
 environment variable OMP_NUM_THREADS=2.

warnings.warn(
 No artists with labels found to put in legend. Note that artists whose label
 start with an underscore are ignored when legend() is called with no argument.



```
[753]: kmeans = KMeans(n_clusters=3, n_init=20, random_state=42)

kmeans.fit(X_pca)

# Get predicted cluster label for each point
y_kmeans = kmeans.predict(X_pca)

# Get the centers of the predicted clusters
centers = kmeans.cluster_centers_

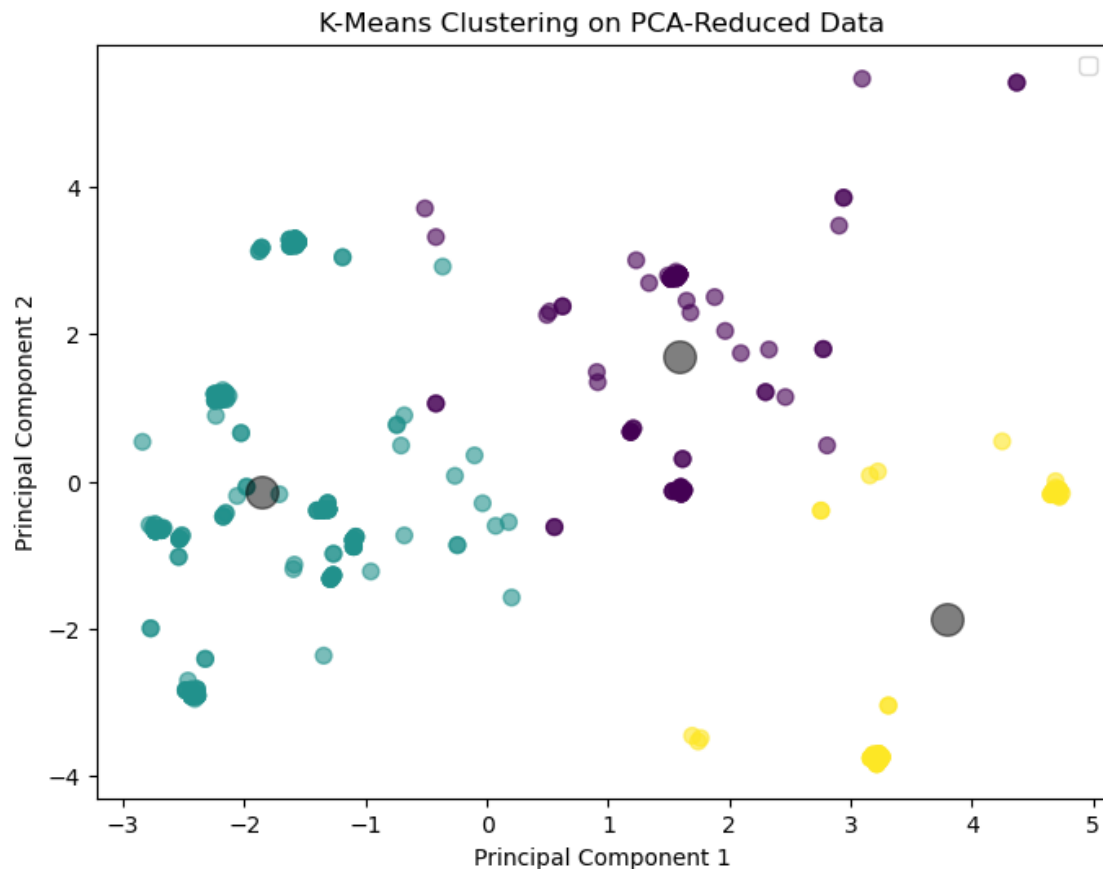
# Plot the clustering result
plt.figure(figsize=(8,6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans, cmap='viridis', s=50, alpha=0.
↪6)
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5)
plt.title("K-Means Clustering on PCA-Reduced Data")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
```

```
plt.show()
```

C:\Users\molin\anaconda3\Lib\site-packages\sklearn\cluster_kmeans.py:1446:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=2.

```
warnings.warn(
```

No artists with labels found to put in legend. Note that artists whose label
start with an underscore are ignored when legend() is called with no argument.



```
[754]: print("Shape of PCA components (transposed):", pca.components_.T.shape)
print("Number of numeric features:", len(df_clust.columns))
```

Shape of PCA components (transposed): (25, 12)

Number of numeric features: 25

```
[755]: X = df_clust.select_dtypes(include=['float64', 'int64'])
```

```
loadings = pd.DataFrame(
    pca.components_.T,
```

```

        columns=[f'PC{i+1}' for i in range(pca.n_components_)],
        index=df_clust.columns # matches the 25 features
    )

loadings_2D = loadings.iloc[:, :2]
print(loadings_2D)

```

	PC1	PC2
Gender	0.135301	-0.269779
Age	0.211969	-0.338651
Sleep Duration	-0.166423	-0.367094
Quality of Sleep	-0.147200	-0.423192
Physical Activity Level	0.115326	-0.166198
Stress Level	0.120588	0.391588
BMI Category	0.357580	0.048109
Heart Rate	0.199393	0.270121
Daily Steps	0.070995	-0.086443
Systolic_BP	0.369282	-0.087487
Diastolic_BP	0.380709	-0.107149
Occ_Accountant	-0.117208	-0.032632
Occ_Doctor	-0.134559	0.219625
Occ_Engineer	-0.153041	-0.188606
Occ_Lawyer	-0.064320	-0.035297
Occ_Manager	-0.000927	0.004182
Occ_Nurse	0.300472	-0.181838
Occ_Sales Representative	0.055928	0.090555
Occ_Salesperson	0.080035	0.190368
Occ_Scientist	0.032349	0.073866
Occ_Software Engineer	-0.006276	0.059883
Occ_Teacher	0.058803	0.013409
SleepDisorder_Insomnia	0.127375	0.161413
SleepDisorder_No Disorder	-0.360039	-0.027670
SleepDisorder_Sleep Apnea	0.309791	-0.127094

[]: